PROJECT 1: Earthquake Safety and Emergency Gate Model

A. COMPONENTS REQUIRED

- Arduino UNO
- Vibration Sensor (digital)
- Push Button
- 5V Adapter / Battery
- UNO Shield
- I2C LCD 16x2
- DIP LED(s)

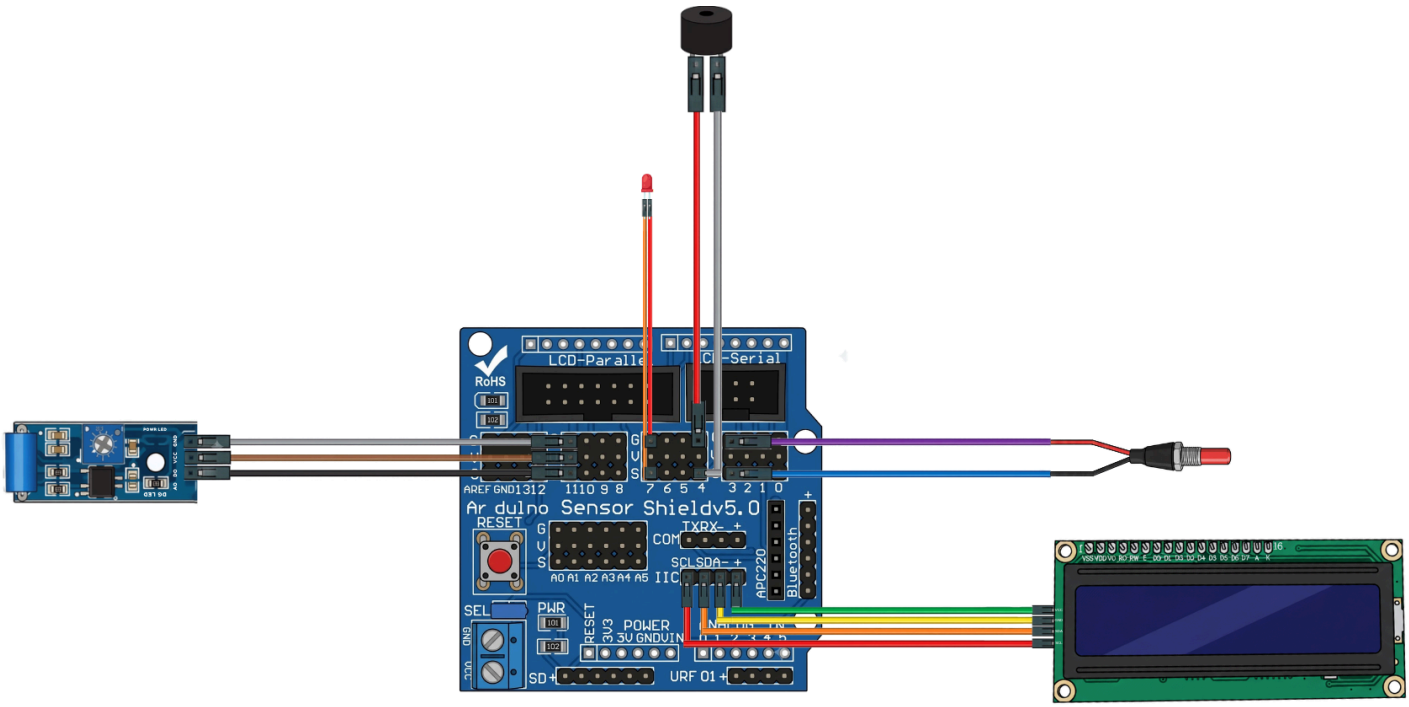
B. PIN CONNECTION TABLE

Component / Signal	Connection
Vibration Sensor OUT	11
Push Button	3
Warning LED	6
Buzzer	4
LCD VCC	5V
LCD GND	GND
LCD SDA	A4
LCD SCL	A5
LCD Address	0x27

C. LIBRARIES REQUIRED

Wire • LiquidCrystal_I2C

D. CIRCUIT DIAGRAM



PROJECT 1: Earthquake Safety and Emergency Gate Model

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

const byte VIB = 11;
const byte BTN = 3;
const byte LED = 6;
const byte BUZZER = 4;

// Time window used to detect sensor instability
const unsigned long CHECK_TIME = 1000;

bool alarmState = false;

int lastSensorState;
unsigned long stateChangeTime;

void setup() {
  pinMode(VIB, INPUT);
  pinMode(BTN, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);

  lcd.init();
  lcd.backlight();

  lcd.setCursor(0, 0);
  lcd.print("EARTHQUAKE SAFE");

  lastSensorState = digitalRead(VIB);
  stateChangeTime = millis();
}

void loop() {

  // Push button resets the system
  if (digitalRead(BTN) == LOW) {
```

```
  alarmState = false;

  noTone(BUZZER);
  digitalWrite(LED, LOW);

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("EARTHQUAKE SAFE");

  // Wait until button is released
  while (digitalRead(BTN) == LOW) {
    delay(10);
  }

  delay(200);
}

int currentSensorState = digitalRead(VIB);

// Detect a change in sensor state
if (currentSensorState != lastSensorState) {

  lastSensorState = currentSensorState;
  stateChangeTime = millis();

  // Sensor is changing → earthquake detected
  alarmState = true;

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("EARTHQUAKE!");
  lcd.setCursor(0, 1);
  lcd.print("EVACUATE CALMLY");
}

// If sensor remains unchanged for CHECK_TIME,
// consider it stable
if (millis() - stateChangeTime >= CHECK_TIME) {

  if (!alarmState) {
    lcd.setCursor(0, 0);
    lcd.print("EARTHQUAKE SAFE ");
    lcd.setCursor(0, 1);
    lcd.print("                ");
```

```
    }
  }

  // Alarm actions
  if (alarmState) {

    // Flash LED
    digitalWrite(LED, (millis() / 200) % 2);

    // Beep buzzer
    if ((millis() / 300) % 2 == 0) {
      tone(BUZZER, 1000);
    } else {
      noTone(BUZZER);
    }

  } else {
    digitalWrite(LED, LOW);
    noTone(BUZZER);
  }

  delay(20);
}
```

PROJECT 2: Smart Parking Slot Indicator

A. COMPONENTS REQUIRED

- Arduino UNO
- 3 x IR Sensors
- I2C LCD 16x2
- 5V Adapter / Battery
- UNO Shield
- Servo Motor
- Buzzer

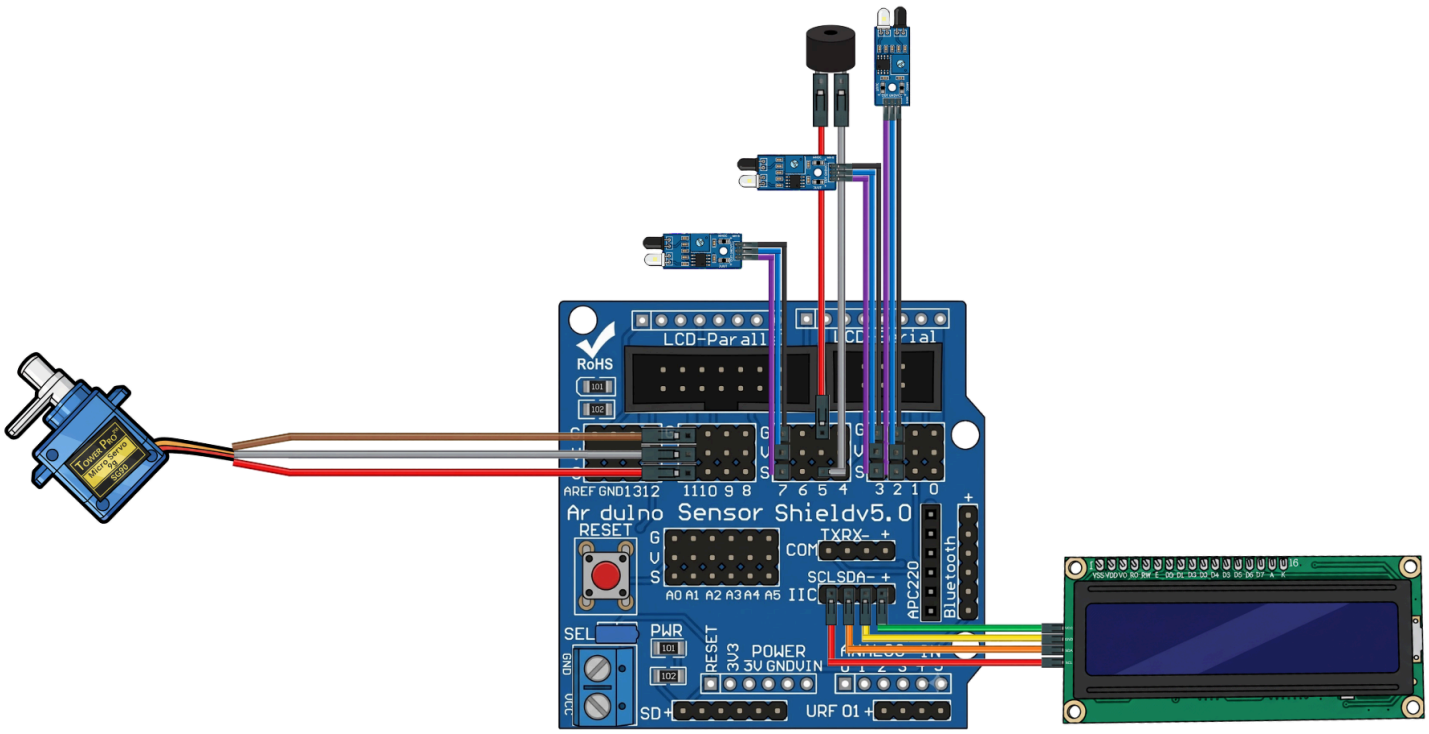
B. PIN CONNECTION TABLE

Component / Signal	Connection
IR Sensor 1 OUT	2
IR Sensor 2 OUT	3
IR Sensor 3 OUT	7
Servo Signal	11
Buzzer	5
LCD VCC	5V
LCD GND	GND
LCD SDA	A4
LCD SCL	A5
LCD Address	0x27

C. LIBRARIES REQUIRED

Wire • LiquidCrystal_I2C • Servo

D. CIRCUIT DIAGRAM



PROJECT 2: Smart Parking Slot Indicator

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo gate;

const byte IR1 = 2;
const byte IR2 = 3;
const byte IR3 = 7;
const byte SERVO = 11;
const byte BUZ = 5;

void setup() {
  pinMode(IR1, INPUT);
  pinMode(IR2, INPUT);
  pinMode(IR3, INPUT);
  pinMode(BUZ, OUTPUT);

  gate.attach(SERVO);
  gate.write(0);

  lcd.init();
  lcd.backlight();
}

void loop() {
  int used = (!digitalRead(IR1)) +
             (!digitalRead(IR2)) +
             (!digitalRead(IR3));

  int freeSlots = 3 - used;

  lcd.setCursor(0, 0);
  lcd.print("FREE SLOTS: ");
  lcd.print(freeSlots);
  lcd.print(" ");

  lcd.setCursor(0, 1);
  lcd.print(freeSlots ? "GATE AVAILABLE " : "PARKING FULL   ");

  gate.write(freeSlots ? 90 : 0);

  digitalWrite(BUZ, freeSlots ? LOW : HIGH);
  delay(200);
}
```

PROJECT 3: Temperature-Controlled Cooling Fan

A. COMPONENTS REQUIRED

- Arduino UNO
- UNO Shield
- DHT11 Sensor
- I2C LCD 16x2
- DC Motor / Fan
- Buzzer
- 5V Adapter / Battery

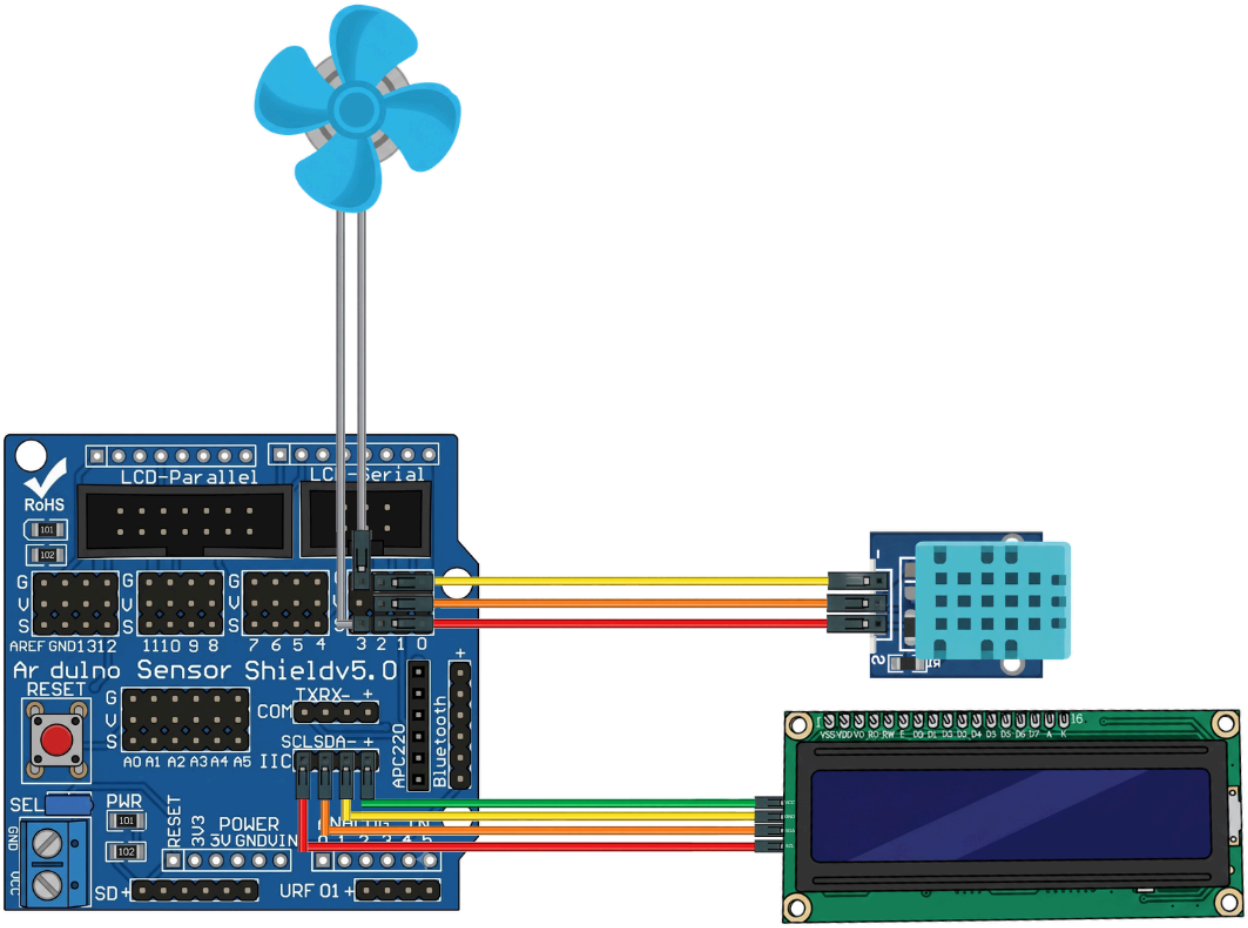
B. PIN CONNECTION TABLE

Component / Signal	Connection
DHT11 DATA	2
DC Motor Control	3
LCD VCC	5V
LCD GND	GND
LCD SDA	A4
LCD SCL	A5
LCD Address	0x27

C. LIBRARIES REQUIRED

Wire • LiquidCrystal_I2C • DHT sensor library

D. CIRCUIT DIAGRAM



PROJECT 3: Temperature-Controlled Cooling Fan

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);
LiquidCrystal_I2C lcd(0x27, 16, 2);

const byte FAN = 3;
const float LIMIT = 30.0;

void setup() {
  pinMode(FAN, OUTPUT);

  dht.begin();
  lcd.init();
  lcd.backlight();
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (isnan(t)) return;
```

```
  bool hot = t >= LIMIT;

  digitalWrite(FAN, hot);

  lcd.setCursor(0, 0);
  lcd.print("T:");
  lcd.print(t, 1);
  lcd.print("C H:");
  lcd.print(h, 0);
  lcd.print("% ");

  lcd.setCursor(0, 1);
  lcd.print(hot ? "FAN: ON         " : "FAN: OFF        ");

  delay(1000);
}
```

PROJECT 4: Automatic Water Heater Controller Using Temperature Sensor

A. COMPONENTS REQUIRED

- Arduino UNO
- UNO Shield
- Waterproof DS18B20 Temperature Sensor
- I2C LCD 16x2
- AC Relay Box
- Bulb / Heater Load
- Buzzer
- 5V Adapter / Battery

B. PIN CONNECTION TABLE

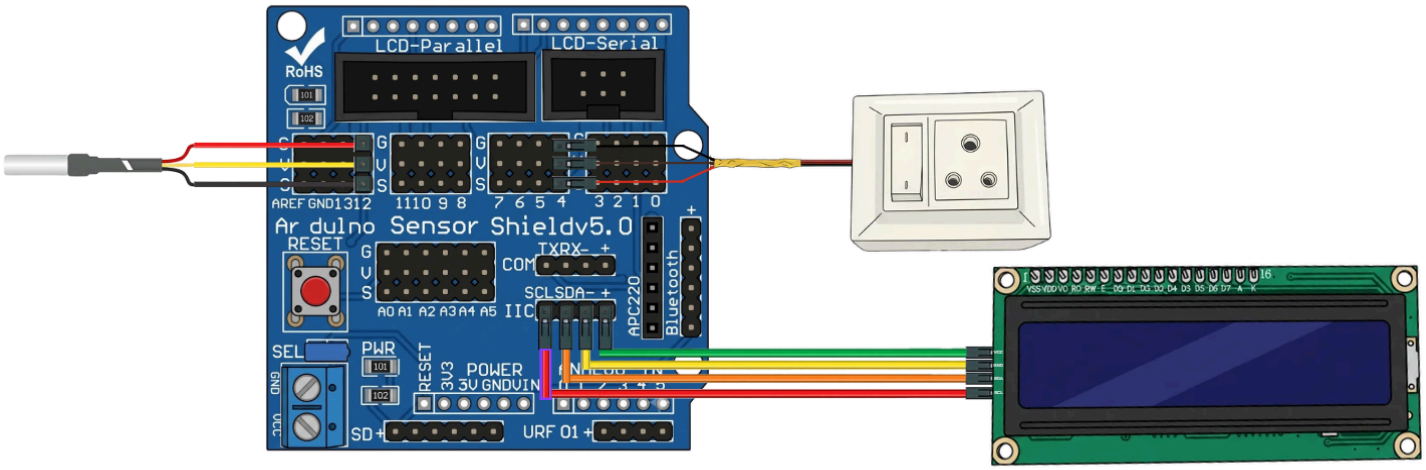
Component / Signal	Connection
DS18B20 DATA	D12
Relay IN	D4
LCD VCC	5V
LCD GND	GND
LCD SDA	A4
LCD SCL	A5
LCD Address	0x27

C. LIBRARIES REQUIRED

Wire • LiquidCrystal_I2C • OneWire • DallasTemperature

NOTE: AC mains must be wired only by a qualified adult/technician. The student circuit controls only the relay input side.

D. CIRCUIT DIAGRAM



PROJECT 4: Automatic Water Heater Controller Using Temperature Sensor

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <OneWire.h>
#include <DallasTemperature.h>
OneWire oneWire(12);
DallasTemperature ds(&oneWire);
LiquidCrystal_I2C lcd(0x27,16,2);
const byte RELAY=4,BUZ=7;
const float SETPOINT=40.0;
void setup()
{
pinMode(RELAY,OUTPUT);
pinMode(BUZ,OUTPUT);
ds.begin();
lcd.init();
lcd.backlight();
}
void loop()
{
 ds.requestTemperatures();
 float t=ds.getTempCByIndex(0);
 bool heat=t<SETPOINT-1;
 digitalWrite(RELAY,heat);
 digitalWrite(BUZ,t>50);
 lcd.setCursor(0,0);
lcd.print("WATER:");
lcd.print(t,1);lcd.print(" C   ");
 lcd.setCursor(0,1);
lcd.print(heat?"HEATER: ON     ":"HEATER: OFF    ");
 delay(1000);
}
```

PROJECT 5: Visitor Token Display

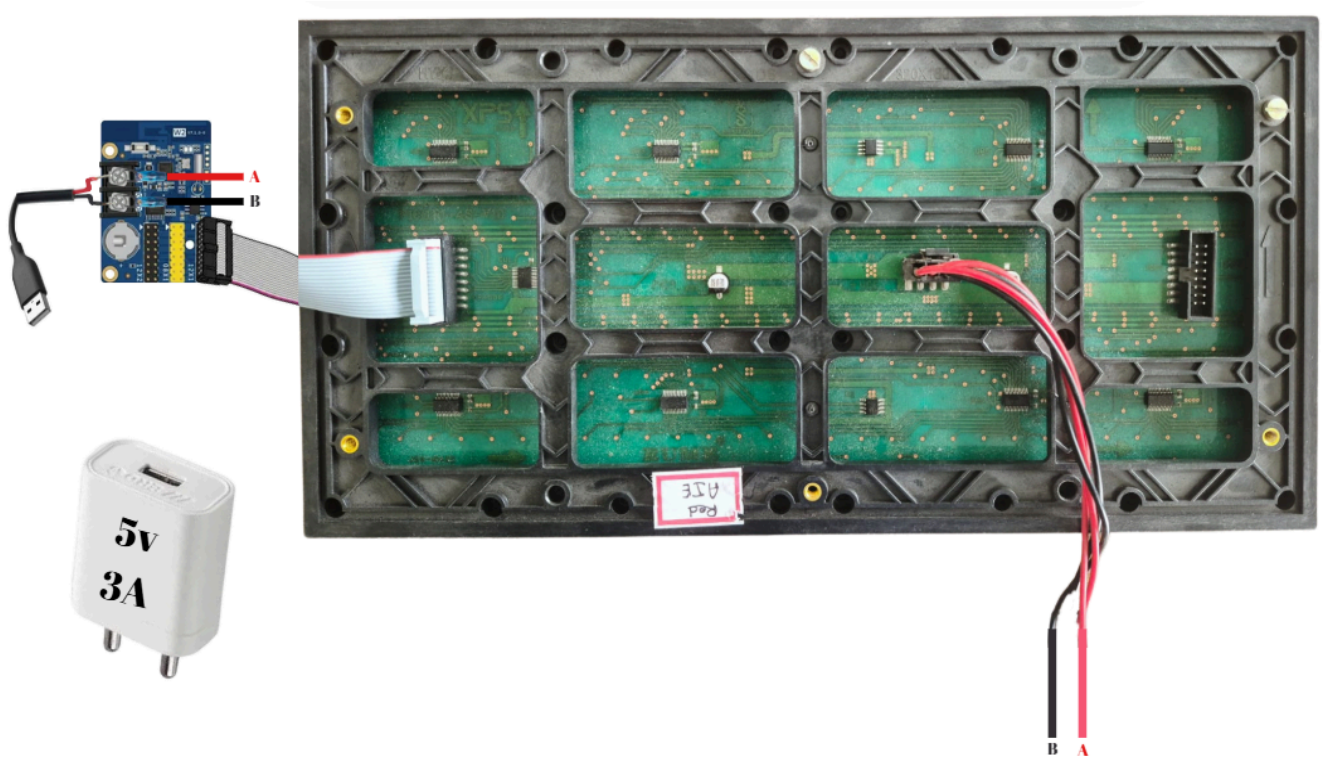
A. COMPONENTS REQUIRED

P10 Driver / Controller

- RGB P10 Display 32x16, 1/8 scan
- 16-pin HUB75 FRC Cable
- P10 Power Cable
- 5V 3A+ Adapter
- Micro USB Cable

B. PIN CONNECTION TABLE

D. CIRCUIT DIAGRAM



PROJECT 6: School-Zone Speed Warning Model

A. COMPONENTS REQUIRED

- ESP32
- 2 x IR Sensors
- Buzzer
- 5V Adapter / Battery
- ESP32 Shield
- I2C LCD 16x2
- DIP LED

B. PIN CONNECTION TABLE

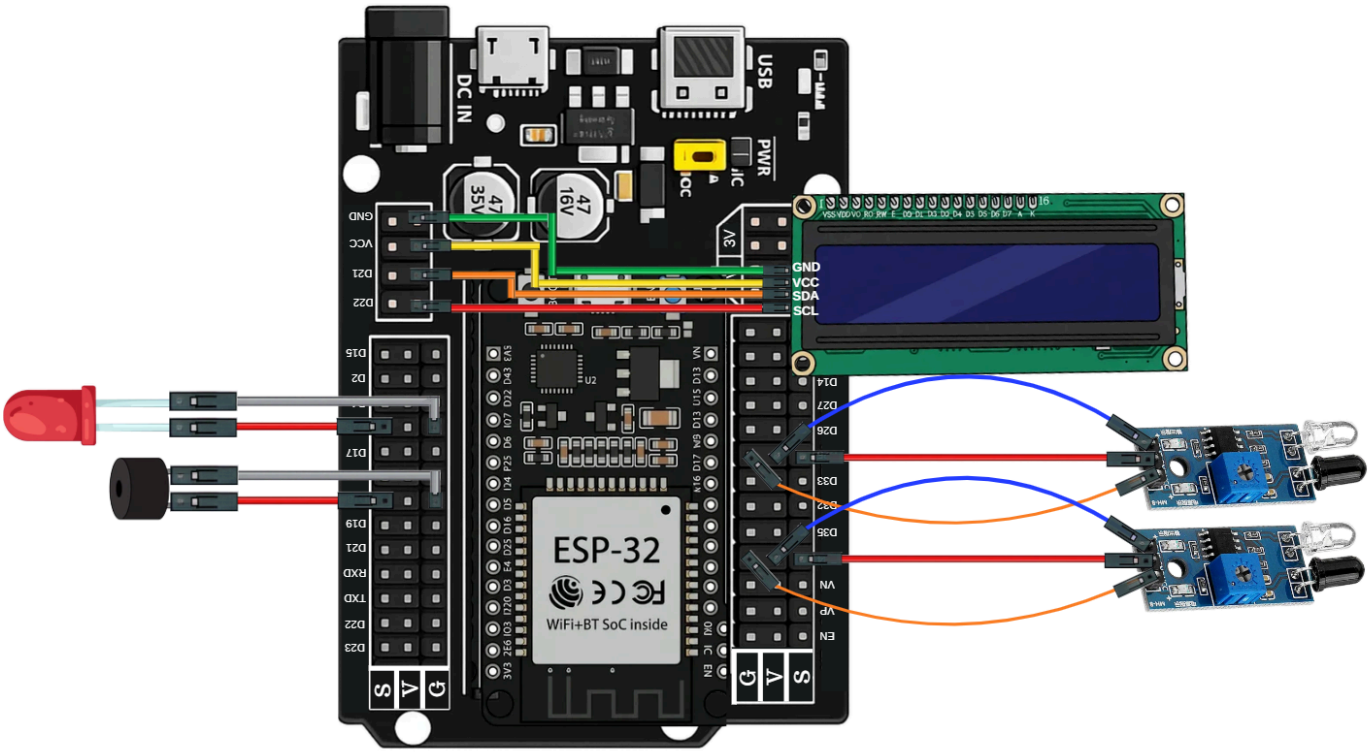
Component / Signal	Connection
IR Sensor 1 OUT	GPIO 34
IR Sensor 2 OUT	GPIO 33
Buzzer	GPIO 25
Warning LED	GPIO 16
LCD VCC	5V
LCD GND	GND
LCD SDA	GPIO 21
LCD SCL	GPIO 22
LCD Address	0x27

C. LIBRARIES REQUIRED

Wire • LiquidCrystal_I2C

NOTE: Set DIST to the actual spacing between the two IR sensors and LIMIT to your model speed limit.

D. CIRCUIT DIAGRAM



PROJECT 6: School-Zone Speed Warning Model

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27,16,2);
const int IR1=34,IR2=25,BUZ=18,LED=16;
 const float DIST=0.20;
 const float LIMIT=10.0;
unsigned long t1=0;
 bool waiting=false,last1=1,last2=1;

Void setup()
{
pinMode(IR1,INPUT);
pinMode(IR2,INPUT);
pinMode(BUZ,OUTPUT);
pinMode(LED,OUTPUT);
lcd.init();
lcd.backlight();
}

void loop(){
 bool a=digitalRead(IR1),b=digitalRead(IR2);
 if(last1&& !a)
{
t1=micros();
waiting=true;
}
 if(waiting&&last2&& !b)
{

float dt=(micros()-t1)/1e6;
float kmh=(DIST/dt)*3.6;
 waiting=false;
  lcd.clear();
lcd.print("SPEED:");
lcd.print(kmh,1);
lcd.print("km/h");
  bool fast=kmh>LIMIT;
digitalWrite(LED,fast);
 if(fast)tone(BUZ,1500,700);
lcd.setCursor(0,1);
lcd.print(fast?"SLOW DOWN!":"SAFE SPEED");
}
 last1=a;
 last2=b;
}
```

PROJECT 7: Door Security System

A. COMPONENTS REQUIRED

- ESP32
- ESP32 Shield
- MC-3T NO/NC Door Sensor
- I2C LCD 16x2
- Buzzer
- Push Button
- DIP LED
- 5V Adapter / Battery

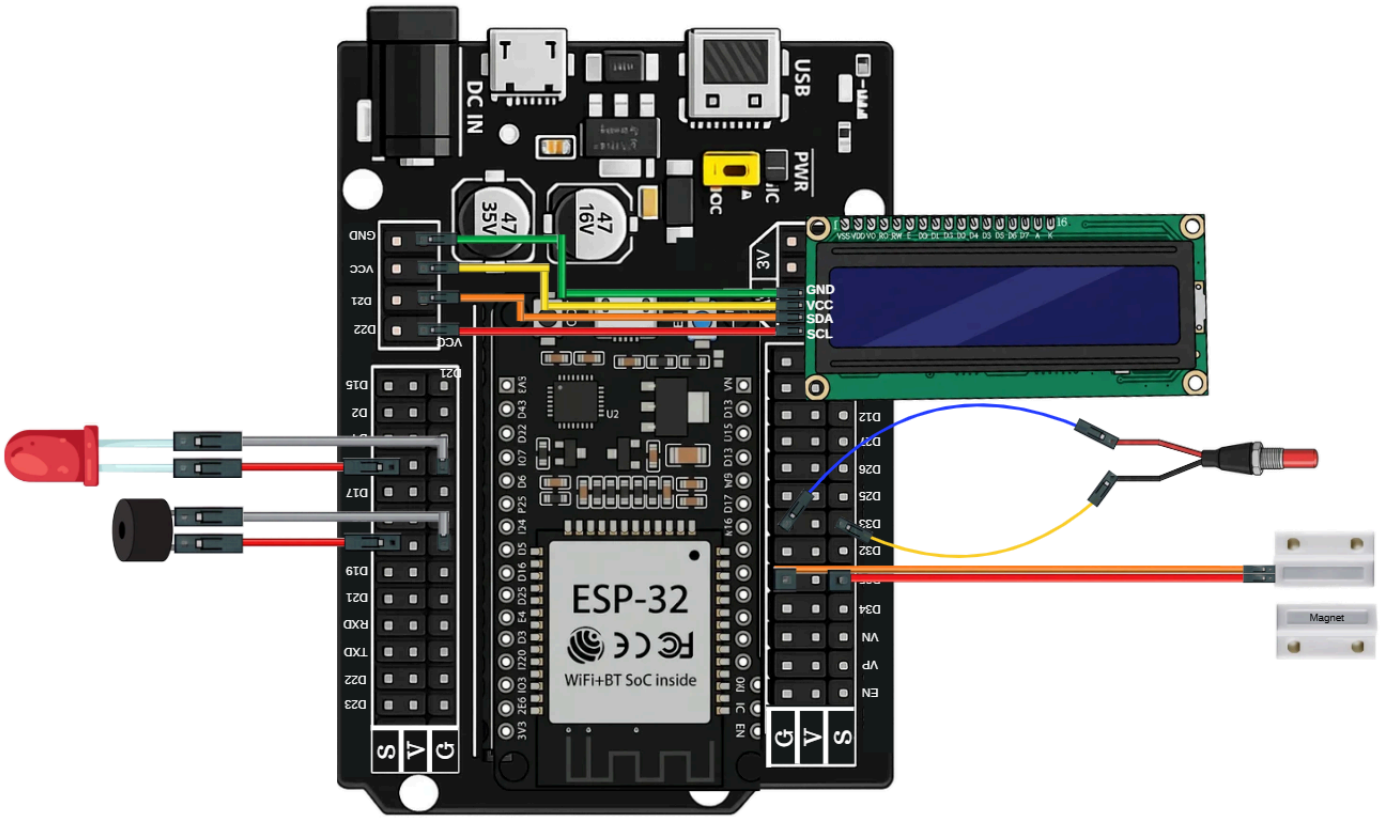
B. PIN CONNECTION TABLE

Component / Signal	Connection
Door Sensor	GPIO 35 to GND (INPUT_PULLUP)
Push Button / Arm	GPIO 33 to GND
Buzzer	GPIO 18
Warning LED	GPIO 16
LCD VCC	5V
LCD GND	GND
LCD SDA	GPIO 21
LCD SCL	GPIO 22
LCD Address	0x27

C. LIBRARIES REQUIRED

Wire • LiquidCrystal_I2C

D. CIRCUIT DIAGRAM



PROJECT 7: Door Security System

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27,16,2);
const int DOOR=35,BTN=33,BUZ=18,LED=16;
bool armed=true,lastBtn=1;
void setup(){pinMode(DOOR,INPUT_PULLUP);
pinMode(BTN,INPUT_PULLUP);
pinMode(BUZ,OUTPUT);
pinMode(LED,OUTPUT);
lcd.init();
lcd.backlight();
}
void loop(){
 bool b=digitalRead(BTN);
 if(lastBtn&&!b)
{armed=!armed;delay(250);
} lastBtn=b;
 bool open=digitalRead(DOOR);
 bool alarm=armed&&open;
 digitalWrite(LED,alarm);
 if(alarm)tone(BUZ,1400);
else
 noTone(BUZ);
 lcd.setCursor(0,0);
lcd.print(armed?"SYSTEM: ARMED   ":"SYSTEM: OFF     ");
 lcd.setCursor(0,1);
lcd.print(open?"DOOR: OPEN      ":"DOOR: CLOSED    ");
 delay(100);
}
```

PROJECT 8: Air Quality, Air Pressure and Temperature Data Logger on Google Sheets

A. COMPONENTS REQUIRED

- ESP32
- ESP32 Shield
- BMP180 Pressure/Temperature Sensor
- I2C LCD 16x2
- Push Button
- Buzzer
- Wi-Fi Connection
- 5V Adapter / Battery

B. PIN CONNECTION TABLE

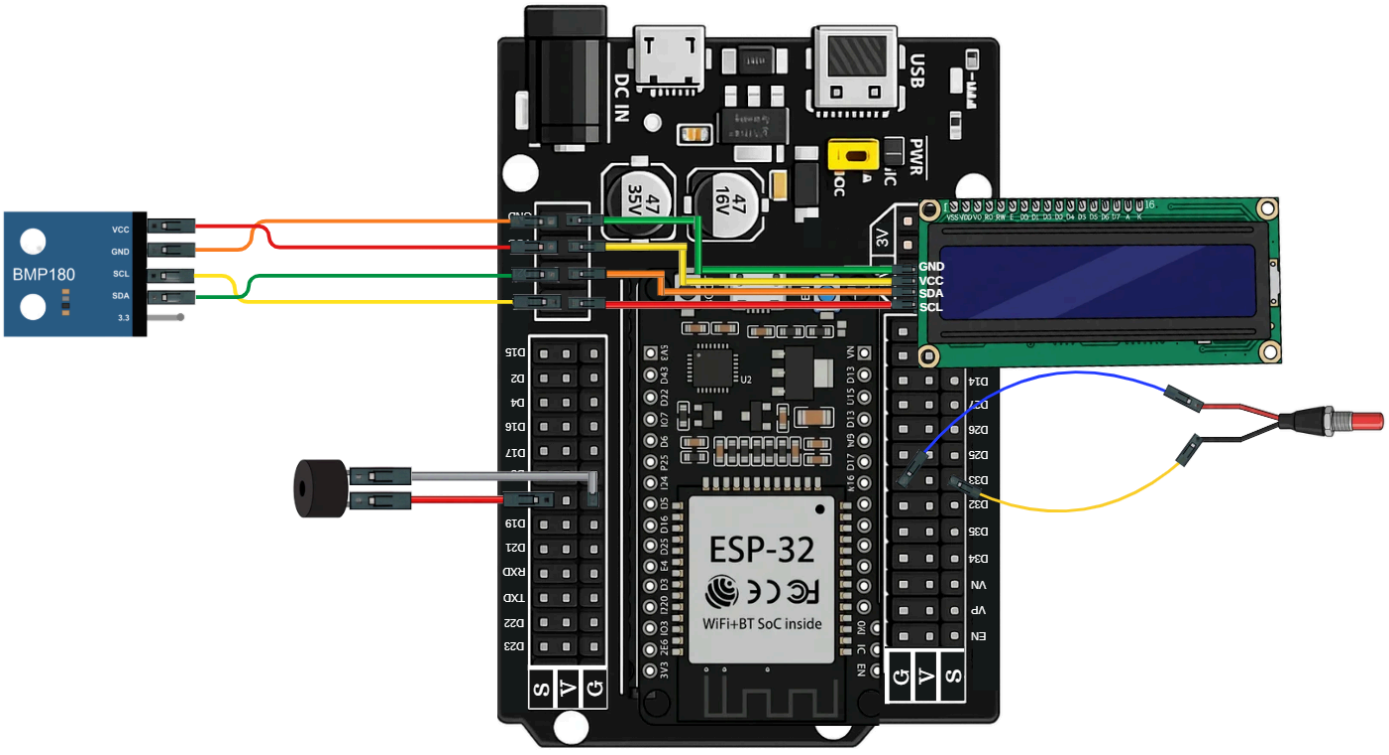
Component / Signal	Connection
BMP180 SDA	GPIO 21
BMP180 SCL	GPIO 22
LCD SDA	GPIO 21
LCD SCL	GPIO 22
Push Button	GPIO 33 to GND
Buzzer	GPIO 18

C. LIBRARIES REQUIRED

Wire • LiquidCrystal_I2C • Adafruit BMP085 Library • WiFi • HTTPClient

NOTE: Your material list does not include an air-quality sensor. This code logs temperature, pressure and altitude from BMP180. Add an MQ135/PM sensor if you need a real air-quality reading.

D. CIRCUIT DIAGRAM



PROJECT 8: Air Quality, Air Pressure and Temperature Data Logger on Google Sheets

CODE

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <WiFiClientSecure.h>
#include <Wire.h>
#include <Adafruit_BMP085.h>
#include <LiquidCrystal_I2C.h>

// ========================================================
// USER SETTINGS
// ========================================================
const char* WIFI_NAME = "Science utsav2";
const char* WIFI_PASSWORD = "Science@2021";

// Paste the URL from Apps Script deployment here
/*
function doGet(e) {

  var sheet = SpreadsheetApp
    .getActiveSpreadsheet()
    .getSheetByName("Data");

  var temperature = e.parameter.temp;
  var pressure = e.parameter.pressure;
  var altitude = e.parameter.altitude;

  sheet.appendRow([
    new Date(),
    temperature,
    pressure,
    altitude,
    "OK"
  ]);

  return ContentService
    .createTextOutput("OK");
}
*/

// Example: https://script.google.com/macros/s/XXXXXXXXXXXX/exec

const char* GOOGLE_SCRIPT_URL =
"https://script.google.com/macros/s/AKfycbzrjLn8670hgCGmejENDMXe4FNY3NdBbGNd9DgDWmEXDp
OvntYkr0KJJhEP4leGn8fC/exec";

// ========================================================
// HARDWARE
// ========================================================
#define SDA_PIN       21
#define SCL_PIN       22
#define BUZZER_PIN    18
#define BUTTON_PIN    33

LiquidCrystal_I2C lcd(0x27, 16, 2);
Adafruit_BMP085 bmp;

// Auto-upload every 30 seconds.
const unsigned long UPLOAD_INTERVAL = 30000;
unsigned long lastUpload = 0;

bool lastButtonState = HIGH;

void beepSuccess() {
  tone(BUZZER_PIN, 1500, 120);
  delay(150);
  tone(BUZZER_PIN, 2200, 120);
}
```

```
void showReading(float temperature, float pressureHpa) {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("T:");
  lcd.print(temperature, 1);
  lcd.print((char)223);
  lcd.print("C");

  lcd.setCursor(0, 1);
  lcd.print("P:");
  lcd.print(pressureHpa, 1);
  lcd.print("hPa");
}

bool sendToGoogleSheets(float temperature, float pressureHpa, float altitudeM) {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("WiFi not connected");
    return false;
  }

  WiFiClientSecure client;
  client.setInsecure(); // Simple classroom HTTPS setup

  HTTPClient http;

  String url = String(GOOGLE_SCRIPT_URL) +
               "?temp=" + String(temperature, 2) +
               "&pressure=" + String(pressureHpa, 2) +
               "&altitude=" + String(altitudeM, 2);

  Serial.println("Sending data...");
  Serial.println(url);

  http.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);

  if (!http.begin(client, url)) {
    Serial.println("HTTP begin failed");
    return false;
  }

  int httpCode = http.GET();
  String response = "";

  if (httpCode > 0) {
    response = http.getString();
  }

  Serial.print("HTTP code: ");
  Serial.println(httpCode);
  Serial.print("Response: ");
  Serial.println(response);

  http.end();

  return (httpCode == 200 || httpCode == 302);
}

void uploadReading() {
  float temperature = bmp.readTemperature();
  float pressureHpa = bmp.readPressure() / 100.0;
  float altitudeM = bmp.readAltitude();

  showReading(temperature, pressureHpa);

  Serial.println("----------------------");
  Serial.print("Temperature: ");
  Serial.print(temperature, 2);
  Serial.println(" C");

  Serial.print("Pressure: ");
  Serial.print(pressureHpa, 2);
  Serial.println(" hPa");
```

```
  Serial.print("Altitude: ");
  Serial.print(altitudeM, 2);
  Serial.println(" m");

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Uploading...");

  bool success = sendToGoogleSheets(temperature, pressureHpa, altitudeM);

  if (success) {
    lcd.setCursor(0, 1);
    lcd.print("Google Sheet OK");
    beepSuccess();
  } else {
    lcd.setCursor(0, 1);
    lcd.print("Upload Failed");
  }

  delay(1500);
  showReading(temperature, pressureHpa);
}

void connectWiFi() {
  lcd.clear();
  lcd.print("Connecting WiFi");

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_NAME, WIFI_PASSWORD);

  int dots = 0;
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
    dots++;

    lcd.setCursor(dots % 16, 1);
    lcd.print(".");
  }

  Serial.println();
  Serial.println("WiFi connected");
  Serial.println(WiFi.localIP());

  lcd.clear();
  lcd.print("WiFi Connected");
  delay(1000);
}

void setup() {
  Serial.begin(115200);

  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);

  Wire.begin(SDA_PIN, SCL_PIN);

  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.print("BMP180 Logger");

  if (!bmp.begin()) {
    lcd.clear();
    lcd.print("BMP180 ERROR");
    Serial.println("BMP180 not detected. Check SDA/SCL and power.");
    while (true) {
      delay(1000);
    }
  }

  connectWiFi();

  // First reading immediately after startup.
```

```
  uploadReading();
  lastUpload = millis();
}

void loop() {
  // Show live reading on LCD.
  float temperature = bmp.readTemperature();
  float pressureHpa = bmp.readPressure() / 100.0;
  showReading(temperature, pressureHpa);

  // Automatic upload every 30 seconds.
  if (millis() - lastUpload >= UPLOAD_INTERVAL) {
    lastUpload = millis();
    uploadReading();
  }

  // Push button = upload immediately.
  bool buttonState = digitalRead(BUTTON_PIN);
  if (lastButtonState == HIGH && buttonState == LOW) {
    delay(30); // debounce
    if (digitalRead(BUTTON_PIN) == LOW) {
      uploadReading();
      lastUpload = millis();
    }
  }

  lastButtonState = buttonState;
  delay(250);
}
```

PROJECT 9: API-Based Weather Monitor System

A. COMPONENTS REQUIRED

- ESP32
- 2.4-inch SPI TFT (ILI9341 assumed)
- Buzzer
- 5V Adapter / Battery
- ESP32 Shield
- 3 x Push Buttons
- Wi-Fi Connection

B. PIN CONNECTION TABLE

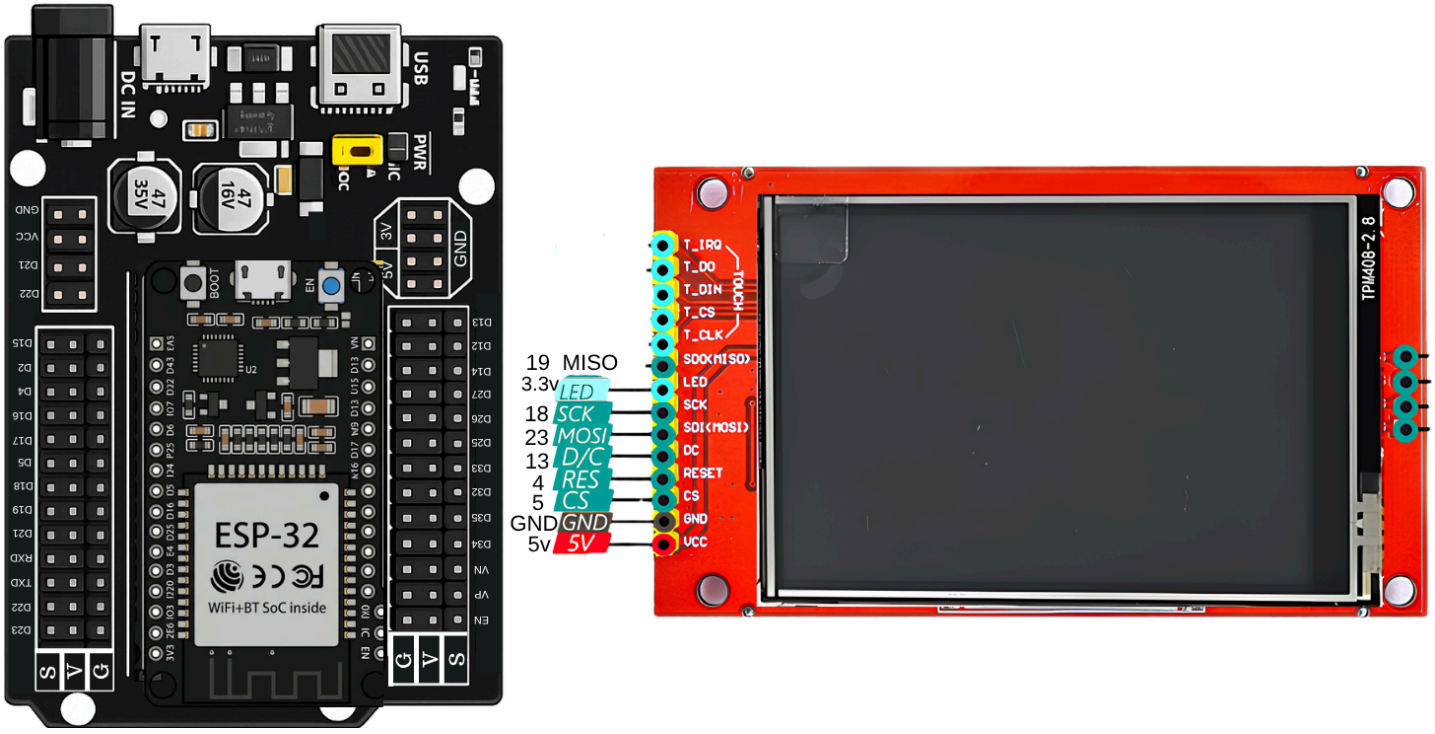
Component / Signal	Connection
TFT VCC	5V or 3.3V as marked on module
TFT GND	GND
TFT SCK	GPIO 18
TFT MOSI	GPIO 23
TFT MISO	GPIO 19
TFT CS	GPIO 5
TFT DC	GPIO 27
TFT RST	GPIO 4
Button 1	GPIO 32 to GND
Button 2	GPIO 33 to GND
Button 3	GPIO 25 to GND
Buzzer	GPIO 26

C. LIBRARIES REQUIRED

WiFi • HTTPClient • ArduinoJson • Adafruit GFX • Adafruit ILI9341

NOTE: Assumes an SPI ILI9341 TFT. If your 2.4-inch TFT uses a different controller, the display library/pins may need to change.

D. CIRCUIT DIAGRAM



PROJECT 9: API-Based Weather Monitor System

CODE

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <WiFiClientSecure.h>
#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ST7789.h>
#include <ArduinoJson.h>

// Wi-Fi credentials
const char* WIFI_SSID = "AQI";
const char* WIFI_PASS = "123456789";

// TFT pins from your module
#define TFT_CS   5
#define TFT_DC   13
#define TFT_RST  4
#define TFT_MOSI 23
#define TFT_MISO 19
#define TFT_CLK  18

Adafruit_ST7789 tft(TFT_CS, TFT_DC, TFT_RST);

struct LocationData {
  String city;
  String region;
  String country;
  float lat;
  float lon;
};

struct AirData {
  int usAqi;
  float pm25;
  float temperature;
  int humidity;
  float windSpeed;
  int weatherCode;
};

LocationData locationData;
AirData airData;

unsigned long lastFetchMs = 0;
const unsigned long REFRESH_MS = 10UL * 60UL * 1000UL;
uint8_t pulsePhase = 0;
bool hasData = false;
uint8_t currentPage = 0;
unsigned long lastPageSwitchMs = 0;
const unsigned long PAGE_SWITCH_MS = 6000UL;

String aqiText(int aqi) {
  if (aqi <= 50) return "Good";
  if (aqi <= 100) return "Moderate";
  if (aqi <= 150) return "Unhealthy SG";
  if (aqi <= 200) return "Unhealthy";
  if (aqi <= 300) return "Very Unhealthy";
  return "Hazardous";
}

uint16_t aqiColor(int aqi) {
  if (aqi <= 50) return ST77XX_GREEN;
  if (aqi <= 100) return ST77XX_YELLOW;
  if (aqi <= 150) return 0xFD20; // orange
  if (aqi <= 200) return ST77XX_RED;
  if (aqi <= 300) return 0x780F; // purple
  return 0x7800; // maroon
}

uint16_t mixColor(uint16_t c1, uint16_t c2, uint8_t weight) {
  uint8_t r1 = (c1 >> 11) & 0x1F;
  uint8_t g1 = (c1 >> 5) & 0x3F;
  uint8_t b1 = c1 & 0x1F;
  uint8_t r2 = (c2 >> 11) & 0x1F;
  uint8_t g2 = (c2 >> 5) & 0x3F;
  uint8_t b2 = c2 & 0x1F;

  uint8_t r = (r1 * (255 - weight) + r2 * weight) / 255;
  uint8_t g = (g1 * (255 - weight) + g2 * weight) / 255;
  uint8_t b = (b1 * (255 - weight) + b2 * weight) / 255;
  return (r << 11) | (g << 5) | b;
}

String fitText(const String& text, int maxChars) {
  if ((int)text.length() <= maxChars) return text;
  return text.substring(0, maxChars - 1) + ".";
}

String tickerText() {
  String msg = "AIR " + aqiText(airData.usAqi) + "  AQI " + String(airData.usAqi);
  msg += "  TEMP " + String(airData.temperature, 1) + "C";
```

```
  msg += "  HUM " + String(airData.humidity) + "%";
  msg += "  WIND " + String(airData.windSpeed, 1) + " km/h";
  msg += "  PM2.5 " + String(airData.pm25, 1) + " ug/m3";
  msg += "  " + fitText(locationData.city, 12) + ", " + fitText(locationData.region, 12);
  return msg;
}

String weatherText(int code) {
  if (code == 0) return "Clear";
  if (code <= 3) return "Cloudy";
  if (code <= 48) return "Fog";
  if (code <= 67) return "Rain";
  if (code <= 77) return "Snow";
  if (code <= 82) return "Showers";
  if (code <= 86) return "Snow Showers";
  if (code <= 99) return "Thunder";
  return "Unknown";
}

uint16_t backdropColor(uint16_t y) {
  uint8_t blend = map(y, 0, 239, 0, 255);
  return mixColor(0x0000, 0x11AA, blend);
}

void drawBackground() {
  for (int y = 0; y < 240; y++) {
    tft.drawFastHLine(0, y, 320, backdropColor(y));
  }

  for (int x = 0; x < 320; x += 24) {
    tft.drawFastVLine(x, 0, 240, 0x0841);
  }

  int glowX1 = 90;
  int glowX2 = 250;
  tft.fillCircle(glowX1, 35, 18, mixColor(ST77XX_CYAN, ST77XX_BLACK, 180));
  tft.fillCircle(glowX2, 200, 24, mixColor(aqiColor(max(0, airData.usAqi)), ST77XX_BLACK, 200));
}

void drawTopBar() {
  tft.fillRoundRect(8, 8, 304, 30, 8, 0x18C3);
  tft.drawRoundRect(8, 8, 304, 30, 8, mixColor(ST77XX_CYAN, ST77XX_WHITE, pulsePhase));
  tft.setTextColor(ST77XX_WHITE, 0x18C3);
  tft.setTextSize(2);
  tft.setCursor(16, 16);
  tft.print("AQI RADAR");

  tft.fillRoundRect(205, 12, 42, 22, 8, WiFi.status() == WL_CONNECTED ? 0x0320 : 0x7800);
  tft.setCursor(212, 17);
  tft.print(WiFi.status() == WL_CONNECTED ? "LIVE" : "OFF");

  int minsLeft = 0;
  if (hasData && millis() > lastFetchMs) {
    unsigned long leftMs = REFRESH_MS > (millis() - lastFetchMs) ? REFRESH_MS - (millis() - lastFetchMs) : 0;
    minsLeft = (leftMs + 59999UL) / 60000UL;
  }
  tft.fillRoundRect(252, 12, 52, 22, 8, 0x0841);
  tft.setCursor(258, 17);
  tft.print(String(minsLeft) + "m");
}

void drawPageDots() {
  for (int i = 0; i < 3; i++) {
    uint16_t color = i == currentPage ? ST77XX_CYAN : 0x52AA;
    tft.fillCircle(145 + i * 14, 26, 4, color);
  }
}

void drawLocationCard() {
  tft.fillRoundRect(10, 48, 132, 128, 12, 0x1082);
  tft.drawRoundRect(10, 48, 132, 128, 12, 0x52AA);

  tft.setTextColor(ST77XX_CYAN, 0x1082);
  tft.setTextSize(2);
  tft.setCursor(20, 60);
  tft.print("LOCATION");

  tft.setTextColor(ST77XX_WHITE, 0x1082);
  tft.setCursor(20, 92);
  tft.print(fitText(locationData.city, 11));
  tft.setCursor(20, 118);
  tft.print(fitText(locationData.region, 11));
  tft.setCursor(20, 144);
  tft.print(fitText(locationData.country, 11));

  tft.setTextSize(1);
  tft.setTextColor(0xBDF7, 0x1082);
  tft.setCursor(20, 160);
  tft.print(String(locationData.lat, 2));
  tft.print(", ");
  tft.print(String(locationData.lon, 2));
}

void drawAqiCard() {
  uint16_t accent = aqiColor(airData.usAqi);
  uint16_t cardColor = mixColor(accent, ST77XX_BLACK, 90);
  uint16_t valueColor = ST77XX_WHITE;
```

```
  if (airData.usAqi > 100) {
    valueColor = ST77XX_YELLOW;
  }
  tft.fillRoundRect(150, 48, 160, 128, 12, cardColor);
  tft.drawRoundRect(150, 48, 160, 128, 12, accent);

  tft.setTextColor(ST77XX_WHITE, cardColor);
  tft.setTextSize(2);
  tft.setCursor(162, 60);
  tft.print("US AQI");

  tft.fillRoundRect(242, 56, 58, 28, 10, mixColor(ST77XX_CYAN, ST77XX_BLACK, 120));
  tft.setTextColor(ST77XX_WHITE, mixColor(ST77XX_CYAN, ST77XX_BLACK, 120));
  tft.setTextSize(2);
  tft.setCursor(251, 64);
  tft.print(String(airData.temperature, 0) + "C");

  tft.setTextSize(5);
  tft.setTextColor(valueColor, cardColor);
  if (airData.usAqi < 10) tft.setCursor(206, 92);
  else if (airData.usAqi < 100) tft.setCursor(192, 92);
  else tft.setCursor(177, 92);
  tft.print(airData.usAqi);

  tft.setTextSize(2);
  tft.setTextColor(ST77XX_WHITE, cardColor);
  tft.setCursor(160, 145);
  tft.print(aqiText(airData.usAqi));

  int barX = 160;
  int barY = 160;
  int segments = 6;
  int lit = constrain(map(airData.usAqi, 0, 300, 1, segments), 1, segments);
  for (int i = 0; i < segments; i++) {
    uint16_t segColor = i < lit ? mixColor(accent, ST77XX_WHITE, (i == lit - 1) ? pulsePhase : 40) : 0x2965;
    tft.fillRoundRect(barX + i * 23, barY, 18, 8, 3, segColor);
  }
}

void drawFooterCard() {
  tft.fillRoundRect(10, 186, 300, 44, 12, 0x1082);
  tft.drawRoundRect(10, 186, 300, 44, 12, 0x52AA);

  tft.fillRoundRect(18, 194, 88, 28, 10, 0x0841);
  tft.setTextColor(ST77XX_CYAN, 0x0841);
  tft.setTextSize(2);
  tft.setCursor(28, 202);
  tft.print("PM2.5");

  tft.fillRoundRect(113, 194, 88, 28, 10, 0x0841);
  tft.setTextColor(ST77XX_WHITE, 0x0841);
  tft.setCursor(124, 202);
  tft.print(String(airData.pm25, 1));

  tft.fillRoundRect(208, 194, 94, 28, 10, 0x0841);
  tft.setTextColor(ST77XX_YELLOW, 0x0841);
  tft.setCursor(220, 202);
  tft.print(String(airData.temperature, 1) + " C");
}

void drawPulse() {
  uint16_t glow = mixColor(aqiColor(airData.usAqi), ST77XX_WHITE, pulsePhase);
  tft.fillCircle(286, 26, 6, glow);
  tft.drawCircle(286, 26, 8, glow);
}

void drawTicker() {
  String msg = tickerText();
  tft.fillRect(0, 230, 320, 10, ST77XX_BLACK);
  tft.setTextColor(ST77XX_WHITE, ST77XX_BLACK);
  tft.setTextSize(1);
  tft.setCursor(6, 232);
  tft.print(fitText(msg, 52));
}

void drawDashboardPage() {
  drawLocationCard();
  drawAqiCard();
  drawFooterCard();
}

void drawWeatherPage() {
  tft.fillRoundRect(10, 48, 300, 180, 12, 0x1082);
  tft.drawRoundRect(10, 48, 300, 180, 12, 0x52AA);

  tft.setTextColor(ST77XX_CYAN, 0x1082);
  tft.setTextSize(2);
  tft.setCursor(20, 60);
  tft.print("WEATHER NOW");

  uint16_t tempCard = mixColor(ST77XX_YELLOW, ST77XX_BLACK, 110);
  tft.fillRoundRect(20, 92, 132, 72, 12, tempCard);
  tft.setTextColor(ST77XX_WHITE, tempCard);
  tft.setCursor(30, 104);
  tft.print("TEMP");
  tft.setTextSize(4);
  tft.setCursor(30, 126);
```

```
  tft.print(String(airData.temperature, 1));
  tft.print("C");

  uint16_t humCard = mixColor(ST77XX_CYAN, ST77XX_BLACK, 120);
  tft.fillRoundRect(168, 92, 132, 72, 12, humCard);
  tft.setTextColor(ST77XX_WHITE, humCard);
  tft.setTextSize(2);
  tft.setCursor(178, 104);
  tft.print("HUMIDITY");
  tft.setTextSize(4);
  tft.setCursor(178, 126);
  tft.print(String(airData.humidity));
  tft.print("%");

  tft.fillRoundRect(20, 176, 132, 42, 10, 0x0841);
  tft.fillRoundRect(168, 176, 132, 42, 10, 0x0841);
  tft.setTextColor(ST77XX_GREEN, 0x0841);
  tft.setTextSize(2);
  tft.setCursor(28, 188);
  tft.print("WIND ");
  tft.print(String(airData.windSpeed, 1));

  tft.setTextColor(ST77XX_WHITE, 0x0841);
  tft.setCursor(176, 188);
  tft.print(weatherText(airData.weatherCode));
}

void drawStatusPage() {
  tft.fillRoundRect(10, 48, 300, 180, 12, 0x1082);
  tft.drawRoundRect(10, 48, 300, 180, 12, 0x52AA);

  tft.setTextColor(ST77XX_CYAN, 0x1082);
  tft.setTextSize(2);
  tft.setCursor(20, 60);
  tft.print("DEVICE STATUS");

  tft.setTextColor(ST77XX_WHITE, 0x1082);
  tft.setCursor(20, 94);
  tft.print("WiFi:");
  tft.setCursor(120, 94);
  tft.print(WiFi.status() == WL_CONNECTED ? "Connected" : "Disconnected");

  tft.setCursor(20, 120);
  tft.print("IP:");
  tft.setCursor(120, 120);
  tft.print(WiFi.localIP());

  tft.setCursor(20, 146);
  tft.print("Signal:");
  tft.setCursor(120, 146);
  tft.print(String(WiFi.RSSI()) + " dBm");

  tft.setCursor(20, 172);
  tft.print("Refresh:");
  tft.setCursor(120, 172);
  tft.print("10 min");

  tft.setCursor(20, 198);
  tft.print("Coords:");
  tft.setCursor(120, 198);
  tft.print(String(locationData.lat, 2));
  tft.print(",");
  tft.print(String(locationData.lon, 2));
}

void drawMessage(const String& title, const String& line1, const String& line2 = "") {
  drawBackground();
  drawTopBar();
  tft.fillRoundRect(18, 56, 284, 112, 14, 0x1082);
  tft.drawRoundRect(18, 56, 284, 112, 14, ST77XX_CYAN);
  tft.setTextWrap(false);
  tft.setTextColor(ST77XX_CYAN, 0x1082);
  tft.setTextSize(3);
  tft.setCursor(34, 70);
  tft.print(title);

  tft.setTextColor(ST77XX_WHITE, 0x1082);
  tft.setTextSize(2);
  tft.setCursor(34, 115);
  tft.print(line1);

  if (line2.length() > 0) {
    tft.setCursor(34, 145);
    tft.print(line2);
  }
}

void animateLoading(const String& step, int frame) {
  drawMessage("ESP32 AQI", step);
  int dotX = 88 + (frame % 4) * 32;
  for (int i = 0; i < 4; i++) {
    uint16_t color = i == (frame % 4) ? ST77XX_YELLOW : 0x39E7;
    tft.fillCircle(88 + i * 32, 190, 8, color);
  }
  tft.drawCircle(dotX, 190, 11, ST77XX_WHITE);
}
```

```
void connectWiFi() {
  drawMessage("WiFi", "Connecting...");
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);

  int tries = 0;
  while (WiFi.status() != WL_CONNECTED && tries < 40) {
    delay(500);
    tries++;
    animateLoading("Connecting...", tries);
    tft.fillRoundRect(34, 205, 200, 18, 8, 0x1082);
    tft.fillRoundRect(34, 205, min(tries * 5, 190), 18, 8, ST77XX_GREEN);
  }
}

bool fetchLocation() {
  HTTPClient http;
  String url = "http://ip-api.com/json/?fields=status,message,country,regionName,city,lat,lon";

  http.begin(url);
  int httpCode = http.GET();
  if (httpCode != 200) {
    http.end();
    return false;
  }

  String payload = http.getString();
  http.end();

  JsonDocument doc;
  DeserializationError err = deserializeJson(doc, payload);
  if (err) {
    return false;
  }

  const char* status = doc["status"] | "";
  if (String(status) != "success") {
    return false;
  }

  locationData.city = String(doc["city"] | "");
  locationData.region = String(doc["regionName"] | "");
  locationData.country = String(doc["country"] | "");
  locationData.lat = doc["lat"] | 0.0;
  locationData.lon = doc["lon"] | 0.0;
  return true;
}

bool fetchAirQuality() {
  WiFiClientSecure client;
  client.setInsecure();

  HTTPClient http;
  String url = "https://air-quality-api.open-meteo.com/v1/air-quality?latitude=" +
               String(locationData.lat, 4) +
               "&longitude=" + String(locationData.lon, 4) +
               "&current=us_aqi,pm2_5";

  http.begin(client, url);
  int httpCode = http.GET();
  if (httpCode != 200) {
    http.end();
    return false;
  }

  String payload = http.getString();
  http.end();

  JsonDocument doc;
  DeserializationError err = deserializeJson(doc, payload);
  if (err) {
    return false;
  }

  airData.usAqi = doc["current"]["us_aqi"] | -1;
  airData.pm25 = doc["current"]["pm2_5"] | -1.0;
  return airData.usAqi >= 0;
}

bool fetchTemperature() {
  WiFiClientSecure client;
  client.setInsecure();

  HTTPClient http;
  String url = "https://api.open-meteo.com/v1/forecast?latitude=" +
               String(locationData.lat, 4) +
               "&longitude=" + String(locationData.lon, 4) +
               "&current=temperature_2m,relative_humidity_2m,wind_speed_10m,weather_code";

  http.begin(client, url);
  int httpCode = http.GET();
  if (httpCode != 200) {
    http.end();
    return false;
  }

  String payload = http.getString();
```

```
  http.end();

  JsonDocument doc;
  DeserializationError err = deserializeJson(doc, payload);
  if (err) {
    return false;
  }

  airData.temperature = doc["current"]["temperature_2m"] | -1000.0;
  airData.humidity = doc["current"]["relative_humidity_2m"] | -1;
  airData.windSpeed = doc["current"]["wind_speed_10m"] | -1.0;
  airData.weatherCode = doc["current"]["weather_code"] | -1;
  return airData.temperature > -999.0;
}

void drawData() {
  drawBackground();
  tft.setTextWrap(false);
  drawTopBar();
  drawPageDots();
  if (currentPage == 0) {
    drawDashboardPage();
  } else if (currentPage == 1) {
    drawWeatherPage();
  } else {
    drawStatusPage();
  }
  drawPulse();
  drawTicker();
}

void refreshData() {
  drawMessage("AQI", "Getting location...");
  if (!fetchLocation()) {
    drawMessage("Error", "Location failed");
    return;
  }

  drawMessage("AQI", "Getting AQI...", locationData.city);
  if (!fetchAirQuality()) {
    drawMessage("Error", "AQI fetch failed");
    return;
  }

  drawMessage("AQI", "Getting Temp...", locationData.city);
  if (!fetchTemperature()) {
    drawMessage("Error", "Temp fetch failed");
    return;
  }

  drawData();
  lastFetchMs = millis();
  hasData = true;
  lastPageSwitchMs = millis();
}

void setup() {
  Serial.begin(115200);

  SPI.begin(TFT_CLK, TFT_MISO, TFT_MOSI, TFT_CS);
  tft.init(240, 320);
  tft.setRotation(1);

  drawMessage("ESP32 AQI", "Booting...");
  connectWiFi();

  if (WiFi.status() == WL_CONNECTED) {
    refreshData();
  } else {
    drawMessage("WiFi", "Connection failed");
  }
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    connectWiFi();
    if (WiFi.status() == WL_CONNECTED) {
      refreshData();
    }
  }

  if (WiFi.status() == WL_CONNECTED && millis() - lastFetchMs > REFRESH_MS) {
    refreshData();
  }

  if (hasData && millis() - lastPageSwitchMs > PAGE_SWITCH_MS) {
    currentPage = (currentPage + 1) % 3;
    lastPageSwitchMs = millis();
    drawData();
  }

  delay(60);
}
```

PROJECT 10: Bridge Tilt and Structural Safety Alert System

A. COMPONENTS REQUIRED

- Arduino UNO
- UNO Shield
- MPU6050 Accelerometer/Gyroscope
- I2C LCD 16x2
- Push Button
- DIP LED(s)
- 5V Adapter / Battery

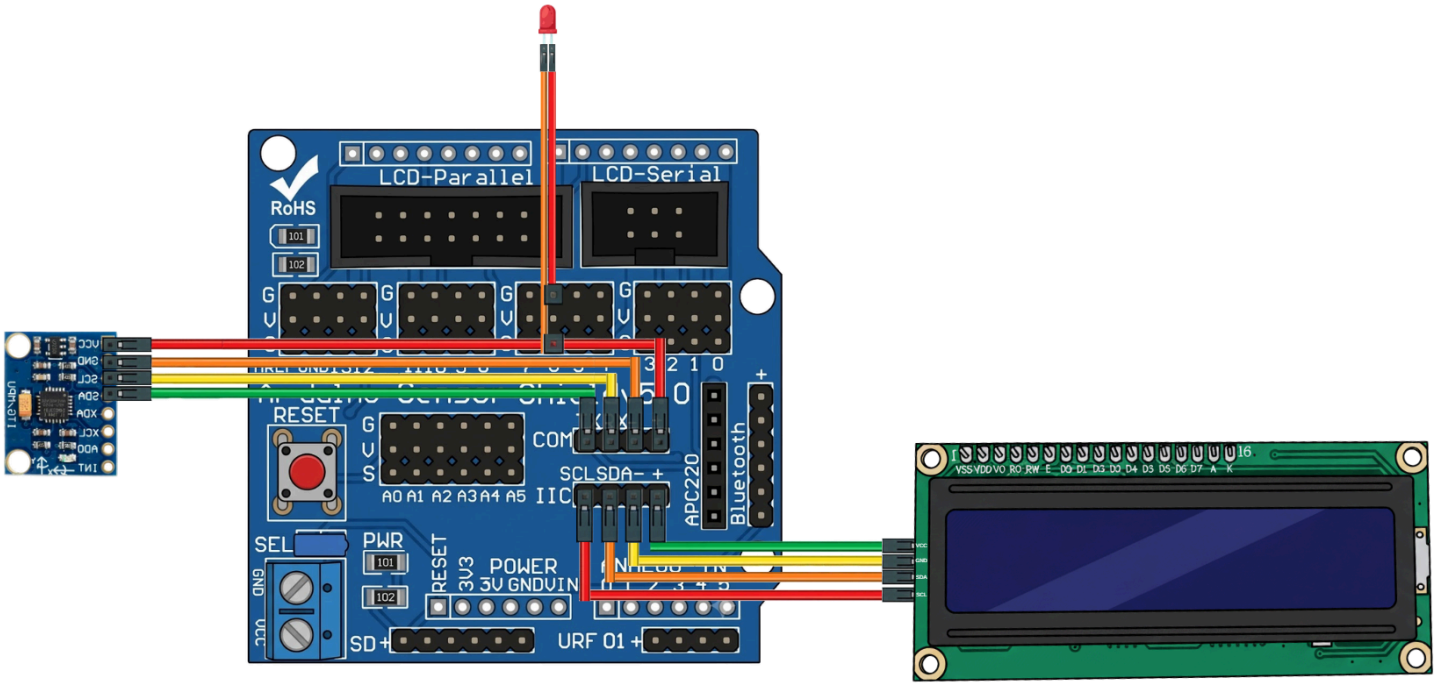
B. PIN CONNECTION TABLE

Component / Signal	Connection
MPU6050 SDA	A4
MPU6050 SCL	A5
LCD SDA	A4
LCD SCL	A5
Warning LED	D6

C. LIBRARIES REQUIRED

Wire • LiquidCrystal_I2C • Adafruit MPU6050 • Adafruit Unified Sensor

D. CIRCUIT DIAGRAM



PROJECT 10: Bridge Tilt and Structural Safety Alert System

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
Adafruit_MPU6050 mpu;

const byte LED = 6;
const float LIMIT = 12.0;

void setup() {
  pinMode(LED, OUTPUT);

  lcd.init();
  lcd.backlight();

  mpu.begin();
}

void loop() {
  sensors_event_t a, g, t;
  mpu.getEvent(&a, &g, &t);

  float roll = atan2(a.acceleration.y, a.acceleration.z) * 180 / PI;

  float pitch = atan2(
    -a.acceleration.x,
    sqrt(a.acceleration.y * a.acceleration.y +
         a.acceleration.z * a.acceleration.z)
  ) * 180 / PI;

  bool danger = abs(roll) > LIMIT || abs(pitch) > LIMIT;

  digitalWrite(LED, danger);

  lcd.setCursor(0, 0);
  lcd.print("R:");
  lcd.print(roll, 0);
  lcd.print(" P:");
  lcd.print(pitch, 0);
  lcd.print("   ");

  lcd.setCursor(0, 1);
  lcd.print(danger ? "STRUCTURE ALERT " : "BRIDGE SAFE     ");

  delay(250);
```

}

PROJECT 11: Automatic Toll Gate and Vehicle Counter

A. COMPONENTS REQUIRED

- Arduino UNO
- UNO Shield
- 3 x IR Sensors
- Servo Motor
- I2C LCD 16x2
- Buzzer
- 5V Adapter / Battery

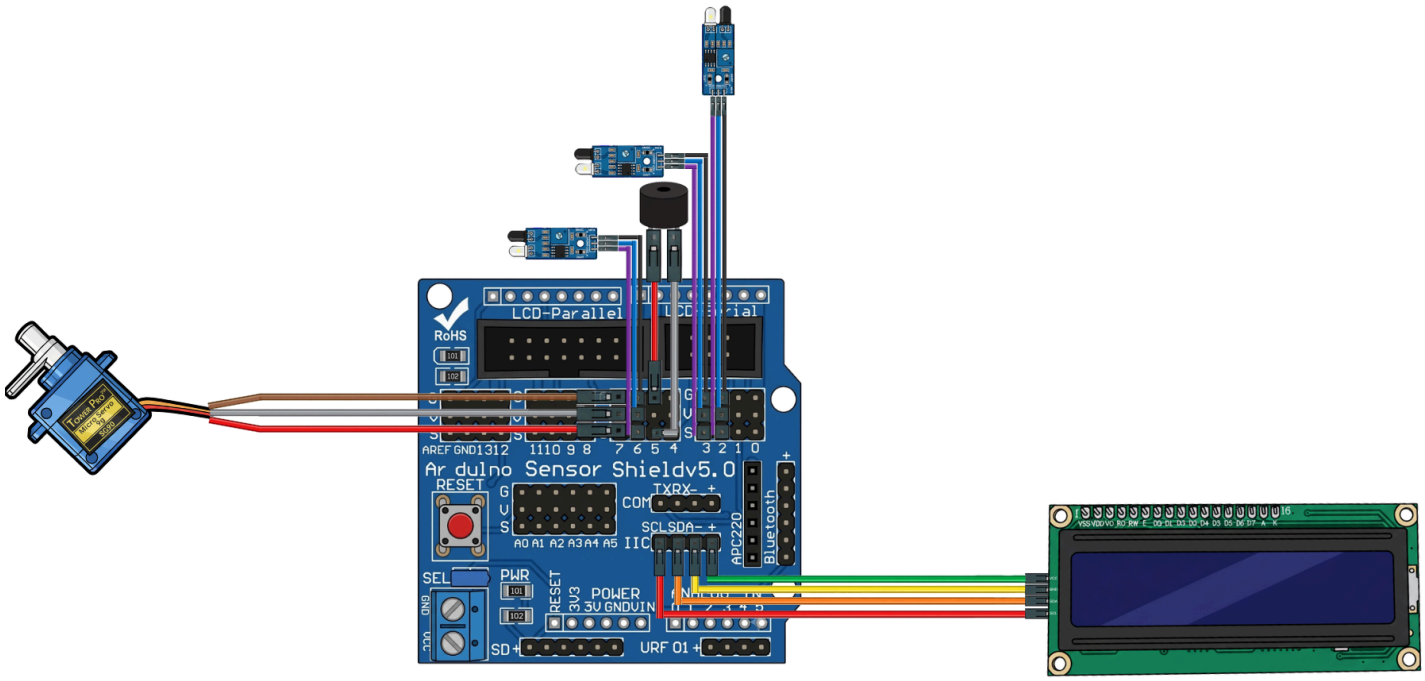
B. PIN CONNECTION TABLE

Component / Signal	Connection
Entry IR	D2
Exit IR	D3
Safety IR	D6
Servo Signal	D7
Buzzer	D5
LCD VCC	5V
LCD GND	GND
LCD SDA	A4
LCD SCL	A5
LCD Address	0x27

C. LIBRARIES REQUIRED

Wire • LiquidCrystal_I2C • Servo

D. CIRCUIT DIAGRAM



PROJECT 11: Automatic Toll Gate and Vehicle Counter

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo gate;
long count = 0;
bool lastIn = 1, lastOut = 1;
const byte IN = 2, OUT = 3, SAFE = 6, SERVO = 7, BUZ = 5;
void setup() {
  pinMode(IN, INPUT);
  pinMode(OUT, INPUT);
  pinMode(SAFE, INPUT);
  pinMode(BUZ, OUTPUT);
  gate.attach(SERVO);
  gate.write(0);
  lcd.init();
  lcd.backlight();
}
void loop() {
  bool a = digitalRead(IN), b = digitalRead(OUT);
  if (lastIn && !a) {
    count++;
    gate.write(90);
    tone(BUZ, 1200, 100);
    delay(500);
  }
  if (lastOut && !b && count > 0) count--;
  if (digitalRead(SAFE) == HIGH) gate.write(0);
  lcd.setCursor(0, 0);
  lcd.print("VEHICLES:");
  lcd.print(count);
  lcd.print("   ");
  lcd.setCursor(0, 1);
  lcd.print("TOLL GATE READY ");
  lastIn = a;
  lastOut = b;
  delay(50);
}
```

PROJECT 12: Smart Greenhouse Ventilation and Heat Alert System

A. COMPONENTS REQUIRED

- Arduino UNO
- UNO Shield
- DHT11 Sensor
- I2C LCD 16x2
- DC Motor / Ventilation Fan
- Buzzer
- 5V Adapter / Battery

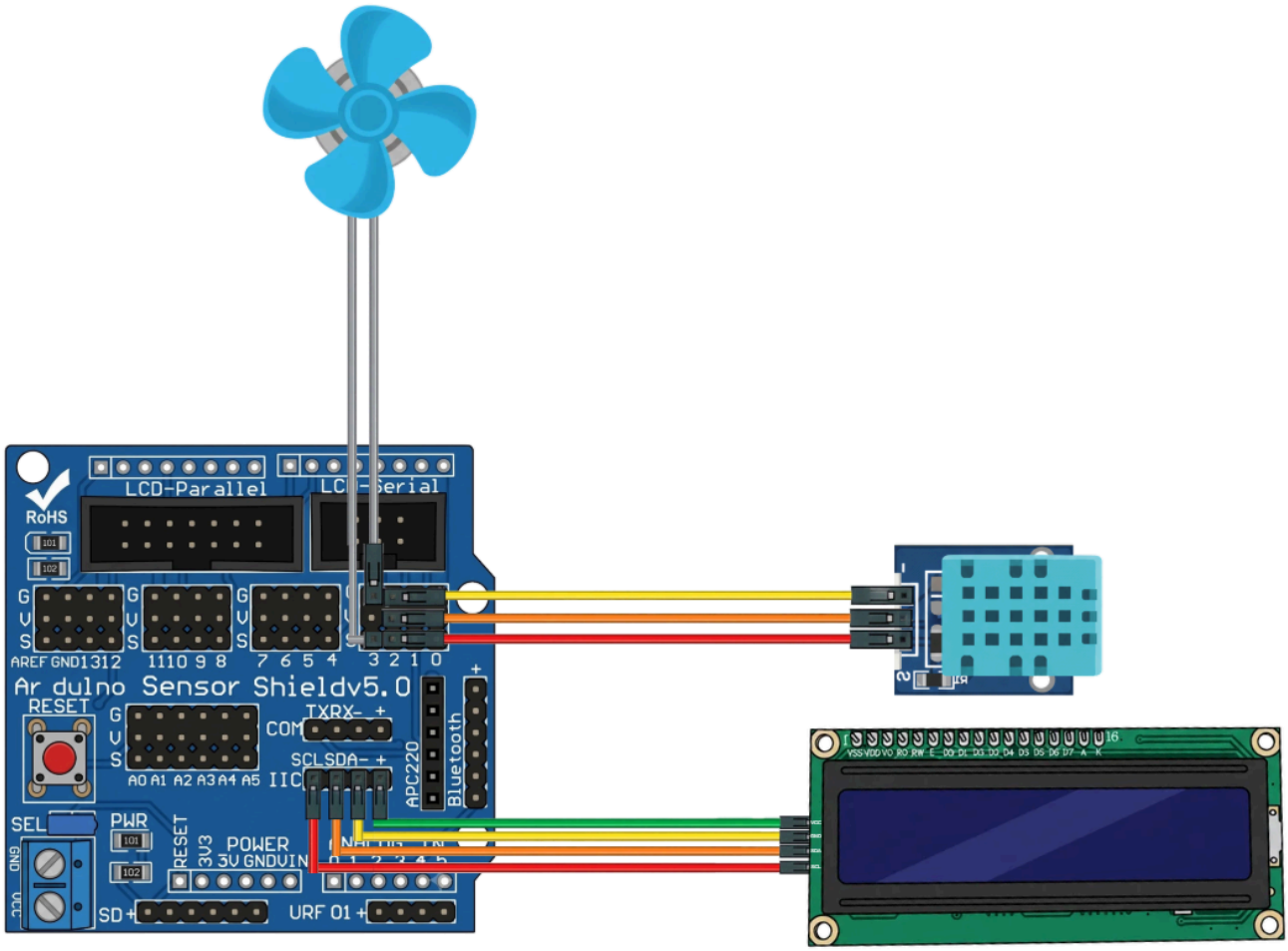
B. PIN CONNECTION TABLE

Component / Signal	Connection
DHT11 DATA	D2
Fan Motor Control	D3 through shield/driver
LCD VCC	5V
LCD GND	GND
LCD SDA	A4
LCD SCL	A5
LCD Address	0x27

C. LIBRARIES REQUIRED

Wire • LiquidCrystal_I2C • DHT sensor library

D. CIRCUIT DIAGRAM



PROJECT 12: Smart Greenhouse Ventilation and Heat Alert System

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);
LiquidCrystal_I2C lcd(0x27, 16, 2);

const byte FAN = 3;
const float LIMIT = 30.0;

void setup() {
  pinMode(FAN, OUTPUT);

  dht.begin();
  lcd.init();
  lcd.backlight();
}

void loop() {
  float t = dht.readTemperature();
```

```
  float h = dht.readHumidity();

  if (isnan(t)) return;

  bool hot = t >= LIMIT;

  digitalWrite(FAN, hot);

  lcd.setCursor(0, 0);
  lcd.print("T:");
  lcd.print(t, 1);
  lcd.print("C H:");
  lcd.print(h, 0);
  lcd.print("% ");

  lcd.setCursor(0, 1);
  lcd.print(hot ? "FAN: ON         " : "FAN: OFF        ");

  delay(1000);
}
```

PROJECT 13: Smart Aquarium Water Temperature Controller

A. COMPONENTS REQUIRED

- Arduino UNO
- UNO Shield
- Waterproof DS18B20 Temperature Sensor
- I2C LCD 16x2
- AC Relay Box
- Bulb / Heater Load
- Buzzer
- 5V Adapter / Battery

B. PIN CONNECTION TABLE

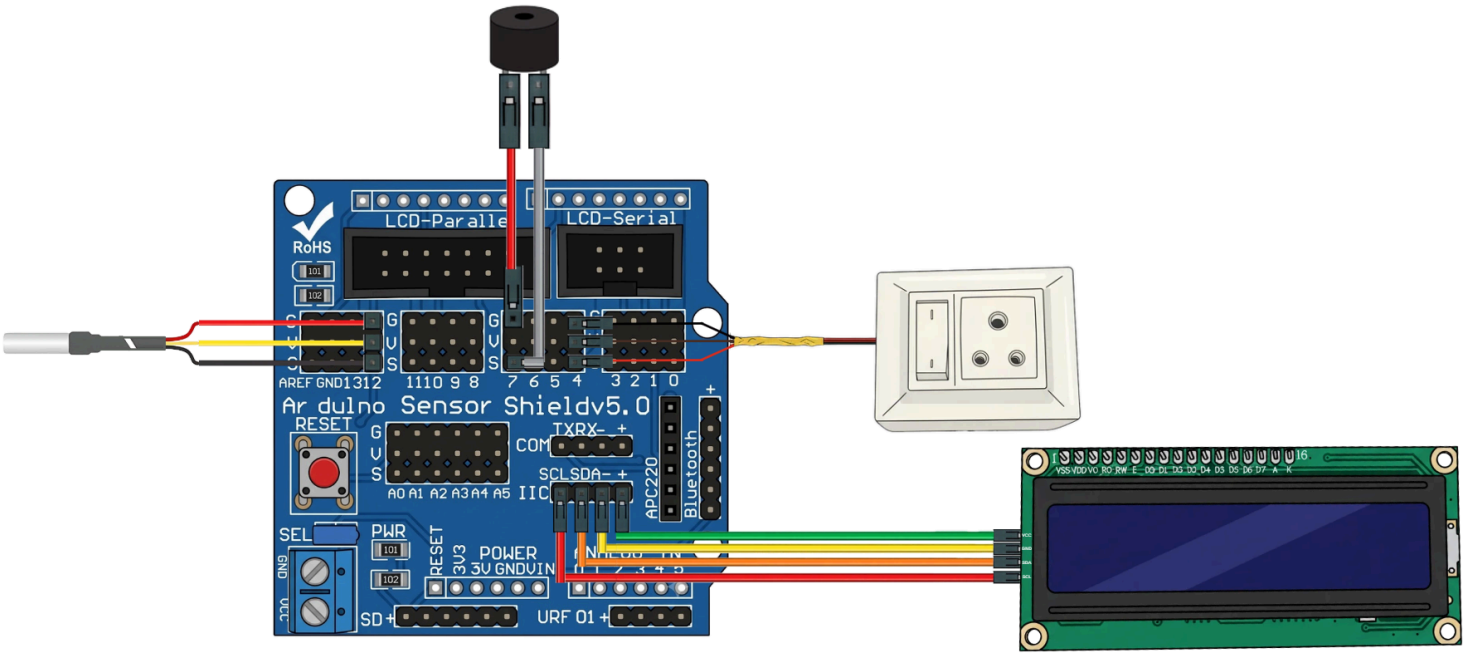
Component / Signal	Connection
DS18B20 DATA	D12
Relay IN	D4
Buzzer	D7
LCD VCC	5V
LCD GND	GND
LCD SDA	A4
LCD SCL	A5
LCD Address	0x27

C. LIBRARIES REQUIRED

Wire • LiquidCrystal_I2C • OneWire • DallasTemperature

NOTE: If the relay controls mains voltage, mains wiring must be done only by a qualified adult/technician.

D. CIRCUIT DIAGRAM



PROJECT 13: Smart Aquarium Water Temperature Controller

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <OneWire.h>
#include <DallasTemperature.h>
OneWire ow(12);
DallasTemperature ds(&ow);
LiquidCrystal_I2C lcd(0x27, 16, 2);
const byte RELAY = 4;
const byte BUZ = 7;
void setup() {
  pinMode(RELAY, OUTPUT);
  pinMode(BUZ, OUTPUT);
  ds.begin();
  lcd.init();
  lcd.backlight();
}
void loop() {
  ds.requestTemperatures();
  float t = ds.getTempCByIndex(0);
  bool heat = t < 24.0;
  digitalWrite(RELAY, heat);
  digitalWrite(BUZ, t < 20 || t > 30);
  lcd.setCursor(0, 0);
  lcd.print("AQUARIUM:");
  lcd.print(t, 1);
  lcd.print("C ");
  lcd.setCursor(0, 1);
  lcd.print(heat ? "HEATER: ON     " : "HEATER: OFF    ");
  delay(1000);
}
```

PROJECT 14: School Announcement and Scrolling Notice Display

A. COMPONENTS REQUIRED

P10 Driver / Controller

- 16-pin HUB75 FRC Cable
- 5V 3A+ Adapter
- P10 Display 32x16, 1/8 scan
- P10 Power Cable
- Micro USB Cable
- Smartphone

B. CIRCUIT DIAGRAM



PROJECT 14: School Announcement and Scrolling Notice Display

APP SETTINGS

Mobile APP – "LedArt" Operating Instructions

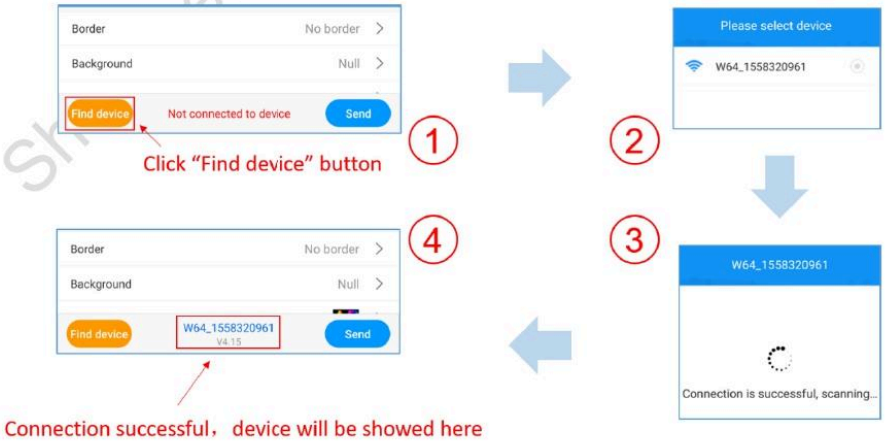
1. Software download

Scanning the QR code downloaded and installed.

Shenzhen Huidu Technology Co., Ltd

2. Connect the control card and modify the Wi-Fi password

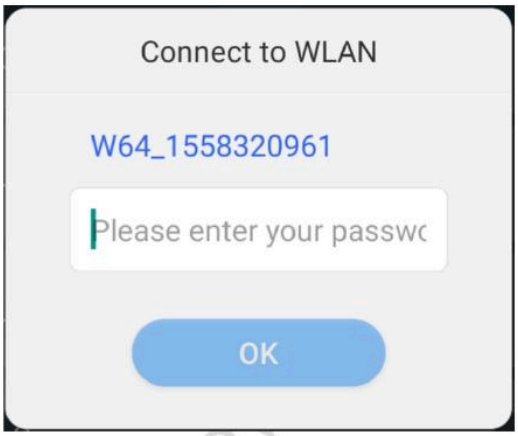
1). Connect the control card: Open the ***LedArt*** application, click the "Find device" button on the lower right corner, select the Wi-Fi signal connect to the control card (the Wi-Fi default password is 88888888).After the connection is successful, the software interface displays the name of the control card.



2). Change password: The initial password is very simple. For the safety of parameters and programs, According to the software prompt, change the password (8~16-digit combination of letters and numbers) and re-enter the new password and connect.



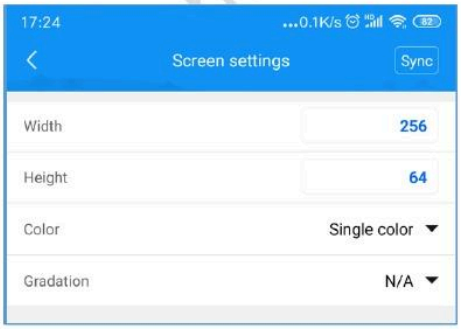
change Password



Enter the password to reconnect

3. Screen Parameters Setting

Click the settings icon in the upper right corner, Select [Screen Parameter Settings] and enter the access password (password: 888) to enter the setting interface, after setting the resolution of the display. The default is the normal P10 parameter (P4.75 selects C2, the chip is 16188B). The single module selects C46, and it is necessary to adjust the scanning frequency, clock frequency, polarity setting and other parameters when the display is abnormal.



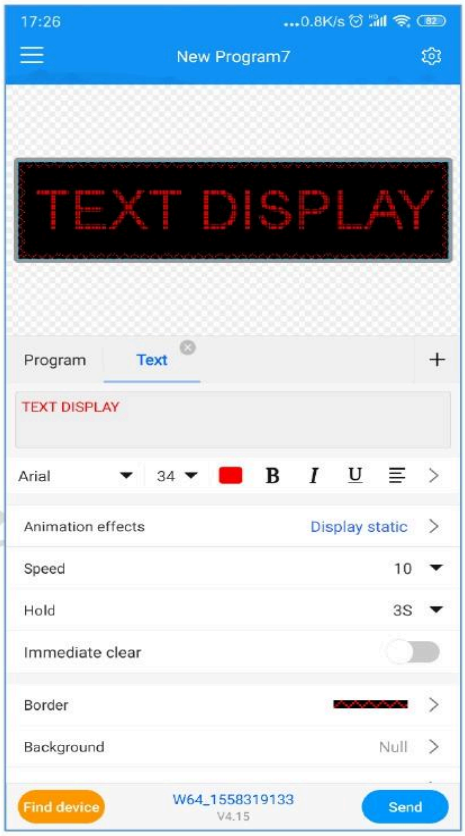


4. Edit and Send Programs

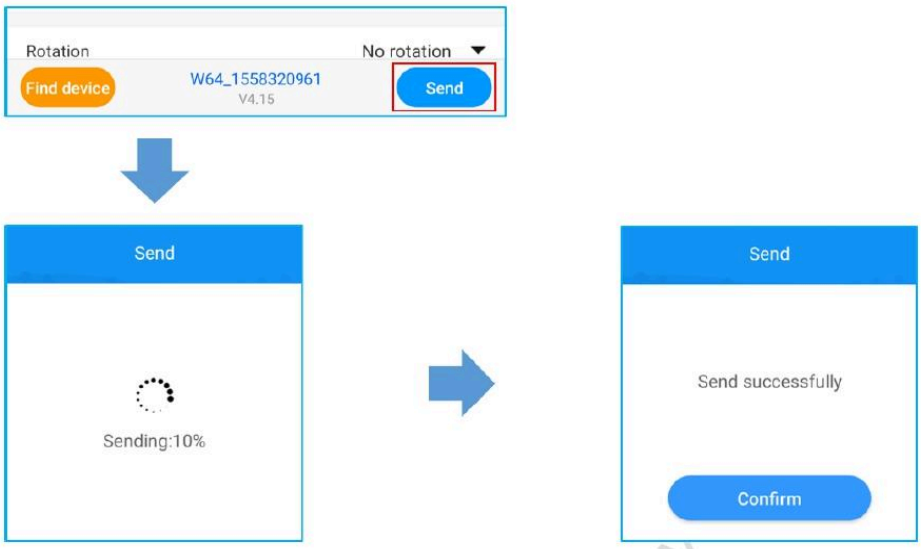
1. Editing program: Click the “+” icon on the right side of the main interface to add the program content, which is optional for text, picture and clock.In the program editing interface, you can set the style of the text as well as display special effects, clock styles and simple pictures.

Note: one display can only add up to two program areas.





2. Sending programs: After edited, click the “Send” button in the lower right corner of the software to update the programs.



PROJECT 15: Remote Controlled System

A. COMPONENTS REQUIRED

- Arduino UNO
- UNO Shield
- IR Remote
- IR Receiver
- NeoPixel RGB Strip
- Buzzer
- 5V Adapter / Battery

B. PIN CONNECTION TABLE

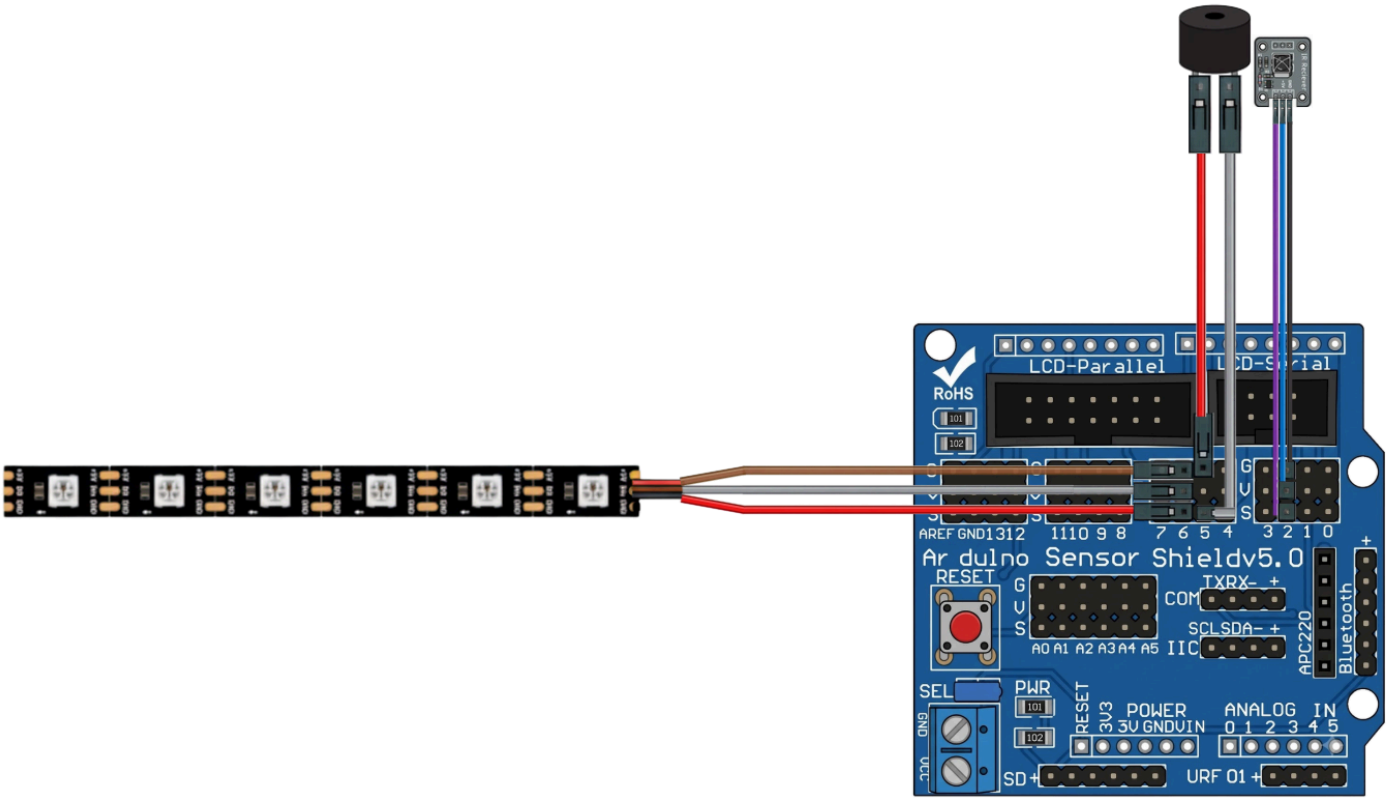
Component / Signal	Connection
IR Receiver OUT	D2
NeoPixel DATA	D6
Buzzer	D5
NeoPixel 5V	External 5V recommended
All Grounds	Common GND

C. LIBRARIES REQUIRED

IRremote • Adafruit NeoPixel

NOTE: IR command values vary by remote. Read the command values in Serial Monitor and replace the example values if required.

D. CIRCUIT DIAGRAM



PROJECT 15: Remote Controlled System

CODE

```
#include <IRremote.hpp>
#include <Adafruit_NeoPixel.h>
#define IR_PIN 11
#define LED_PIN 6
#define NUM_LEDS 30
Adafruit_NeoPixel strip(NUM_LEDS, LED_PIN, NEO_GRB + NEO_KHZ800);
int mode = 0;
#define B1 0x0A
#define B2 0x1B
#define B3 0x1F
#define B4 0x0C
#define B5 0x0D
#define B6 0x0E
#define NEXT 0x09
#define PREV 0x07
#define POWER 0x40

void setup() {
```

```
  Serial.begin(9600);
  strip.begin(); strip.setBrightness(100); strip.show();
  IrReceiver.begin(IR_PIN, ENABLE_LED_FEEDBACK);
}

void loop() {
  readIR();

  if (mode == 0) off();                    // OFF
  else if (mode == 1) color(255,0,0);      // RED
  else if (mode == 2) color(0,255,0);      // GREEN
  else if (mode == 3) color(0,0,255);      // BLUE
  else if (mode == 4) rainbow();           // RAINBOW
  else if (mode == 5) chase();             // CHASE
  else if (mode == 6) blink();             // BLINK
}

void readIR() {
  if (IrReceiver.decode()) {
    if (!(IrReceiver.decodedIRData.flags & IRDATA_FLAGS_IS_REPEAT)) {
      uint8_t c = IrReceiver.decodedIRData.command;
      Serial.println(c, HEX);

      if (c == B1) mode = 1;               // RED
      else if (c == B2) mode = 2;          // GREEN
      else if (c == B3) mode = 3;          // BLUE
      else if (c == B4) mode = 4;          // RAINBOW
      else if (c == B5) mode = 5;          // CHASE
      else if (c == B6) mode = 6;          // BLINK
      else if (c == NEXT) { mode++; if (mode > 6) mode = 1; } // NEXT
      else if (c == PREV) { mode--; if (mode < 1) mode = 6; } // PREVIOUS
      else if (c == POWER) mode = 0;       // OFF
    }
    IrReceiver.resume();
  }
}
void off() {
  strip.clear(); strip.show();
}
void color(byte r, byte g, byte b) {
  for (int i=0; i<NUM_LEDS; i++) strip.setPixelColor(i, r, g, b);
  strip.show();
}
void rainbow() {
```

```
  static uint16_t h = 0;
  for (int i=0; i<NUM_LEDS; i++)
    strip.setPixelColor(i, strip.gamma32(strip.ColorHSV(h + i*65536L/NUM_LEDS)));
  strip.show(); h += 256; delay(20);
}

void chase() {
  static byte s = 0;
  strip.clear();
  for (int i=s; i<NUM_LEDS; i+=3) strip.setPixelColor(i,255,255,255);
  strip.show(); s=(s+1)%3; delay(100);
}
void blink() {
  static bool state = 0;
  static unsigned long t = 0;
  if (millis()-t > 400) {
    t=millis(); state=!state;
    if (state) color(255,0,255);            // PINK
    else off();                             // OFF
  }
}
```

PROJECT 16: Refrigerator or Medicine Cabinet Door-Open Alert

A. COMPONENTS REQUIRED

- ESP32
- ESP32 Shield
- MC-3T NO/NC Door Sensor
- I2C LCD 16x2
- Buzzer
- Push Button
- DIP LED
- 5V Adapter / Battery

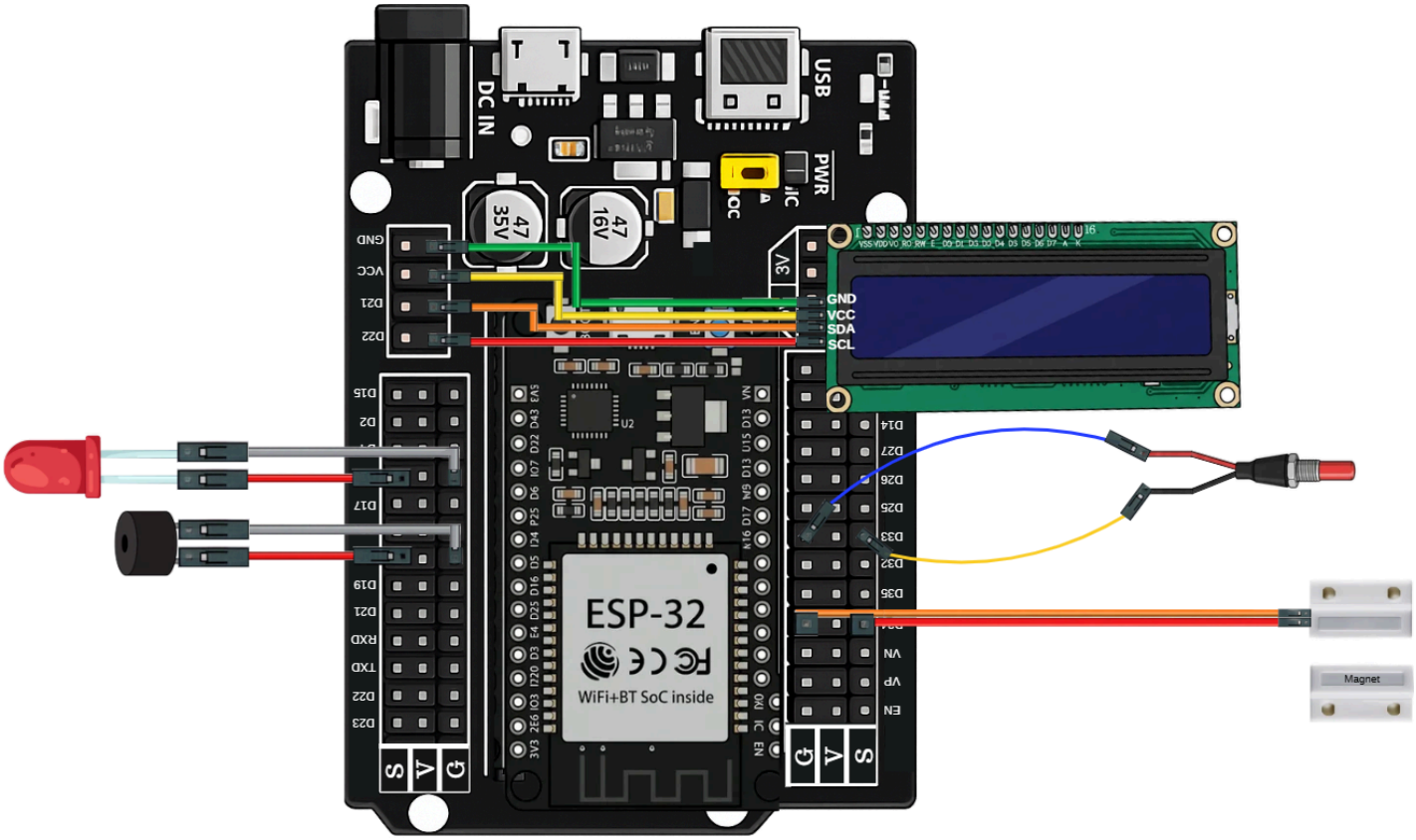
B. PIN CONNECTION TABLE

Component / Signal	Connection
Door Sensor	GPIO 34 to GND (INPUT_PULLUP)
Reset/Mute Button	GPIO 33 to GND
Buzzer	GPIO 18
Warning LED	GPIO 16
LCD VCC	5V
LCD GND	GND
LCD SDA	GPIO 21
LCD SCL	GPIO 22
LCD Address	0x27

C. LIBRARIES REQUIRED

Wire • LiquidCrystal_I2C

D. CIRCUIT DIAGRAM



PROJECT 16: Refrigerator or Medicine Cabinet Door-Open Alert

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
const int DOOR = 34, BTN = 33, BUZ = 18, LED = 16;
unsigned long openedAt = 0;
bool wasOpen = false, muted = false;
void setup() {
  pinMode(DOOR, INPUT_PULLUP);
  pinMode(BTN, INPUT_PULLUP);
  pinMode(BUZ, OUTPUT);
  pinMode(LED, OUTPUT);
  lcd.init();
  lcd.backlight();
}
void loop() {
  bool open = digitalRead(DOOR);
  if (open && !wasOpen) {
    openedAt = millis();
    muted = false;
  }
```

```
  if (!open) {
    openedAt = 0;
    muted = false;
  }
  if (!digitalRead(BTN)) {
    muted = true;
    delay(250);
  }
  wasOpen = open;
  unsigned long sec = open ? (millis() - openedAt) / 1000 : 0;
  bool alarm = open && sec >= 10 && !muted;
  digitalWrite(LED, alarm);
  if (alarm) tone(BUZ, 1500);
  else noTone(BUZ);
  lcd.setCursor(0, 0);
  lcd.print(open ? "DOOR OPEN      " : "DOOR CLOSED    ");
  lcd.setCursor(0, 1);
  lcd.print("TIME:");
  lcd.print(sec);
  lcd.print(" sec     ");
  delay(100);

}
```

PROJECT 17: Smart Trekking Altitude and Weather Data Logger

A. COMPONENTS REQUIRED

- ESP32
- ESP32 Shield
- BMP180 Pressure/Temperature Sensor
- I2C LCD 16x2
- Push Button
- Buzzer
- 5V Adapter / Battery

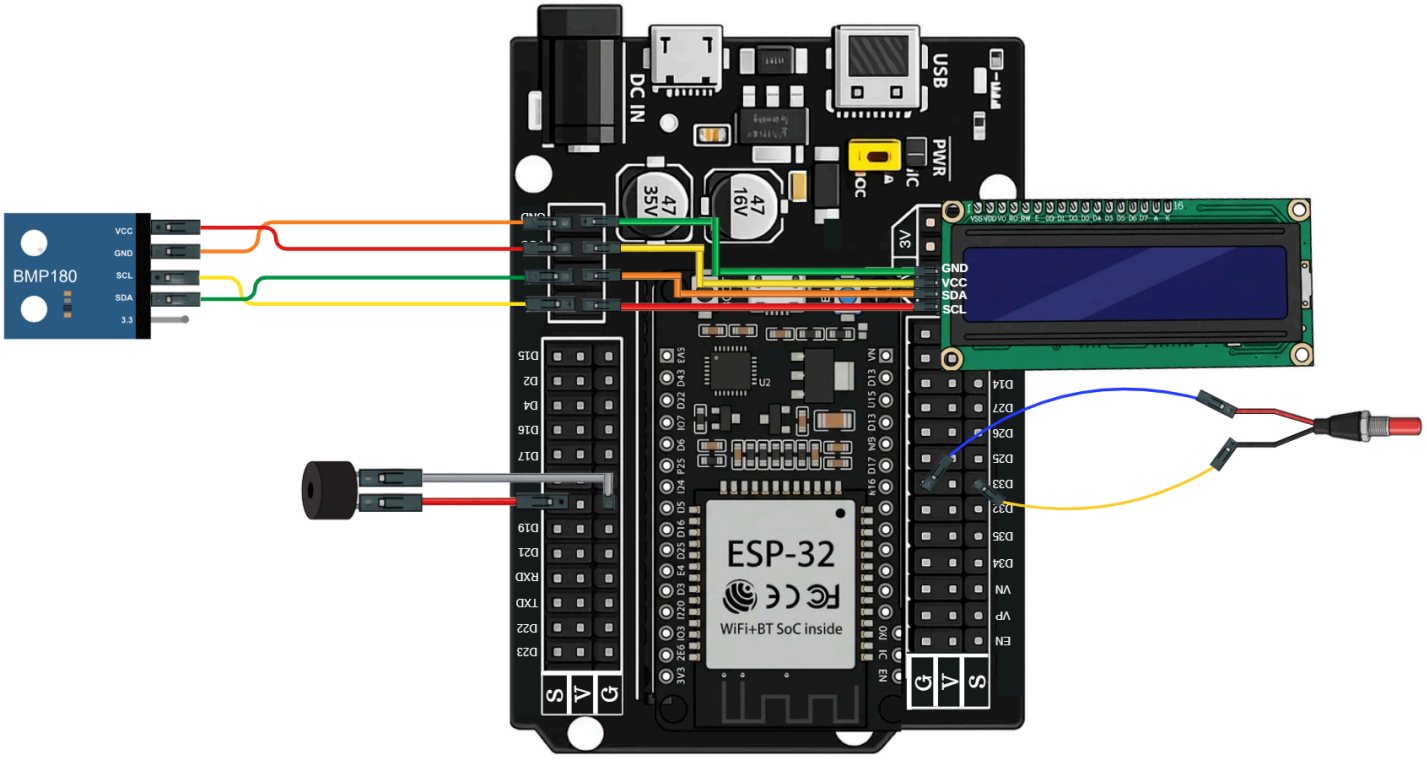
B. PIN CONNECTION TABLE

Component / Signal	Connection
BMP180 SDA	GPIO 21
BMP180 SCL	GPIO 22
LCD SDA	GPIO 21
LCD SCL	GPIO 22
Push Button	GPIO 33 to GND
Buzzer	GPIO 18

C. LIBRARIES REQUIRED

Wire • LiquidCrystal_I2C • Adafruit BMP085 Library

D. CIRCUIT DIAGRAM



PROJECT 17: Smart Trekking Altitude and Weather Data Logger

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Adafruit_BMP085.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
Adafruit_BMP085 bmp;
const int BTN = 33, BUZ = 18;
float maxAlt = -9999;
void setup() {
  pinMode(BTN, INPUT_PULLUP);
  pinMode(BUZ, OUTPUT);
  lcd.init();
  lcd.backlight();
  bmp.begin();
}
void loop() {
  float t = bmp.readTemperature(), p = bmp.readPressure() / 100.0, alt =
bmp.readAltitude();
  if (alt > maxAlt) maxAlt = alt;
  lcd.setCursor(0, 0);
  lcd.print("ALT:");
  lcd.print(alt, 1);
  lcd.print("m   ");
  lcd.setCursor(0, 1);
  lcd.print("T:");
  lcd.print(t, 1);
  lcd.print(" P:");
  lcd.print(p, 0);
  lcd.print(" ");
  if (!digitalRead(BTN)) {
    lcd.clear();
    lcd.print("MAX ALT:");
    lcd.print(maxAlt, 1);
    lcd.setCursor(0, 1);
    lcd.print("TREK DATA");
    tone(BUZ, 1000, 100);
    delay(1500);
  }
  delay(500);
```

}

PROJECT 18: API-Based City NEWS DISPLAY

A. COMPONENTS REQUIRED

- ESP32
- ESP32 Shield
- 2.4-inch SPI TFT (ILI9341 assumed)
- 3 x Push Buttons
- Buzzer
- Wi-Fi Connection
- 5V Adapter / Battery

B. PIN CONNECTION TABLE

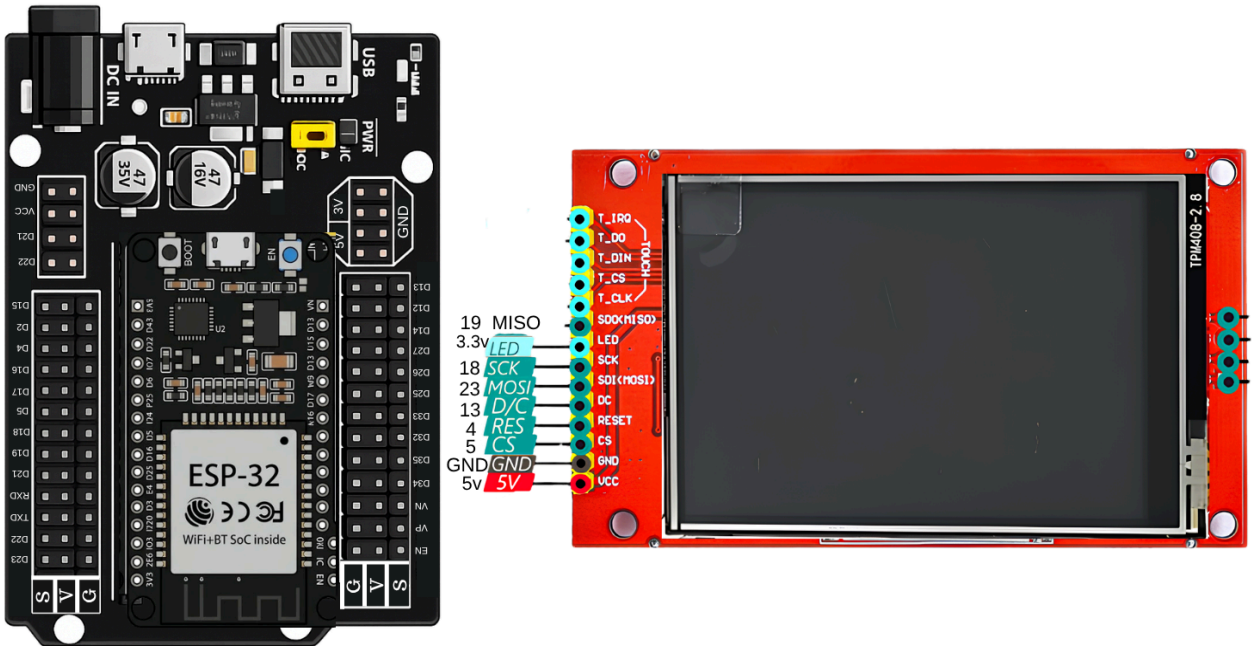
Component / Signal	Connection
TFT VCC	5V or 3.3V as marked on module
TFT GND	GND
TFT SCK	GPIO 18
TFT MOSI	GPIO 23
TFT MISO	GPIO 19
TFT CS	GPIO 5
TFT DC	GPIO 27
TFT RST	GPIO 4

C. LIBRARIES REQUIRED

WiFi • HTTPClient • ArduinoJson • Adafruit GFX • Adafruit ILI9341

NOTE: AQI uses OpenWeather Air Pollution API (index 1-5). Assumes an SPI ILI9341 TFT.

D. CIRCUIT DIAGRAM



PROJECT 18: API-Based NEWS

CODE

PROJECT 19: Smart Flood Level and Drainage Alert System

A. COMPONENTS REQUIRED

- ESP32
- Water Level Sensor
- I2C LCD 16x2
- 5V Adapter / Battery
- ESP32 Shield
- Servo Motor (Drainage Gate)
- Buzzer

B. PIN CONNECTION TABLE

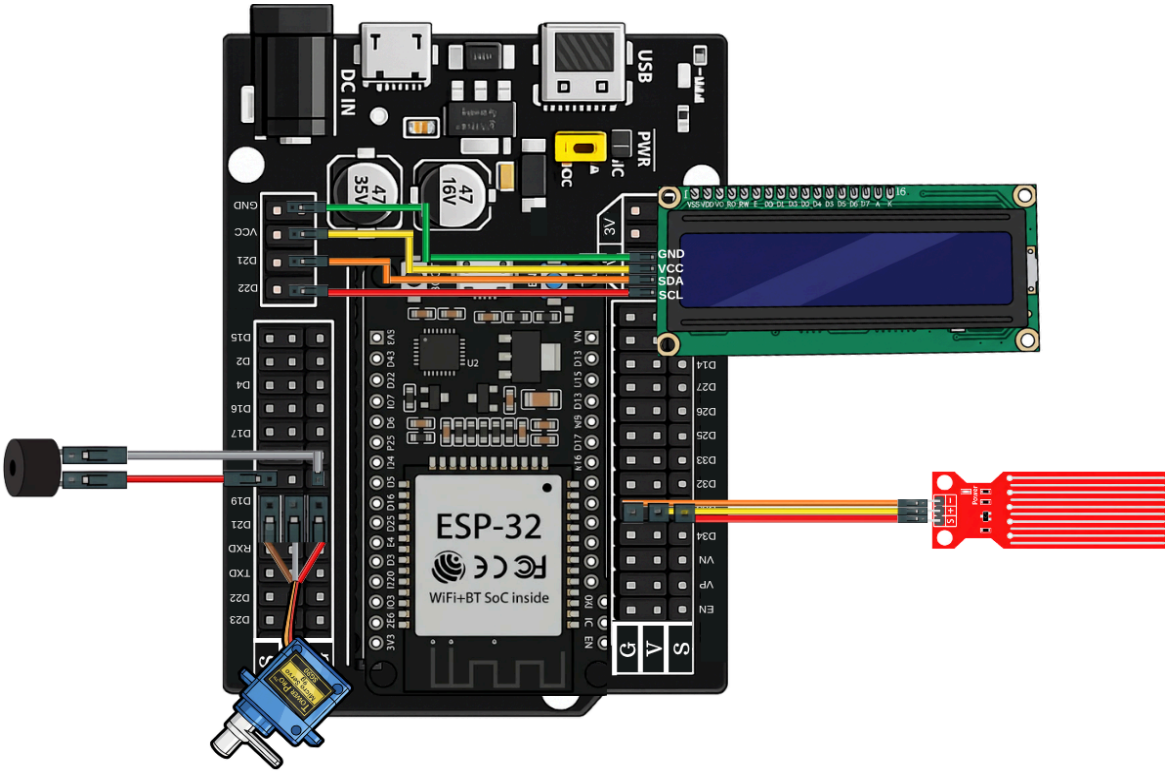
Component / Signal	Connection
Water Level Sensor AO	GPIO 34
Servo Signal	GPIO 19
Buzzer	GPIO 18
LCD VCC	5V
LCD GND	GND
LCD SDA	GPIO 21
LCD SCL	GPIO 22
LCD Address	0x27

C. LIBRARIES REQUIRED

Wire • LiquidCrystal_I2C • ESP32Servo

NOTE: Calibrate ALERT after checking the actual dry and wet analog readings of your water-level sensor.

D. CIRCUIT DIAGRAM


DC IN
USB
PWR
47
35V
47
16V
GND
VCC
SDA
SCL
ESP-32
WiFi+BT SoC inside

PROJECT 19: Smart Flood Level and Drainage Alert System

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <ESP32Servo.h>
LiquidCrystal_I2C lcd(0x27,16,2);
Servo gate;
const int WATER=34,SERVO=19,BUZ=18;
const int ALERT=2600;

void setup()
{
pinMode(BUZ,OUTPUT);
gate.attach(SERVO);
gate.write(0);
lcd.init();lcd.backlight();
}
void loop()
{
 int raw=analogRead(WATER);
int pct=map(raw,0,4095,0,100);
pct=constrain(pct,0,100);
bool flood=raw>=ALERT;
gate.write(flood?90:0);
if(flood)tone(BUZ,1400);
else noTone(BUZ);
 lcd.setCursor(0,0);
lcd.print("WATER:");
lcd.print(pct);lcd.print("%      ");
lcd.setCursor(0,1);
lcd.print(flood?"DRAIN GATE OPEN ":"LEVEL NORMAL    ");
delay(300);
}
```

PROJECT 20: Smart Classroom Light and Fan Automation System

A. COMPONENTS REQUIRED

- ESP32
- ESP32 Shield
- PIR Motion Sensor
- LDR Light Sensor Module
- DHT11 Sensor
- DC Motor / Fan
- I2C LCD 16x2
- Buzzer
- 2 x Push Buttons
- DIP LED(s)
- 5V Adapter / Battery

B. PIN CONNECTION TABLE

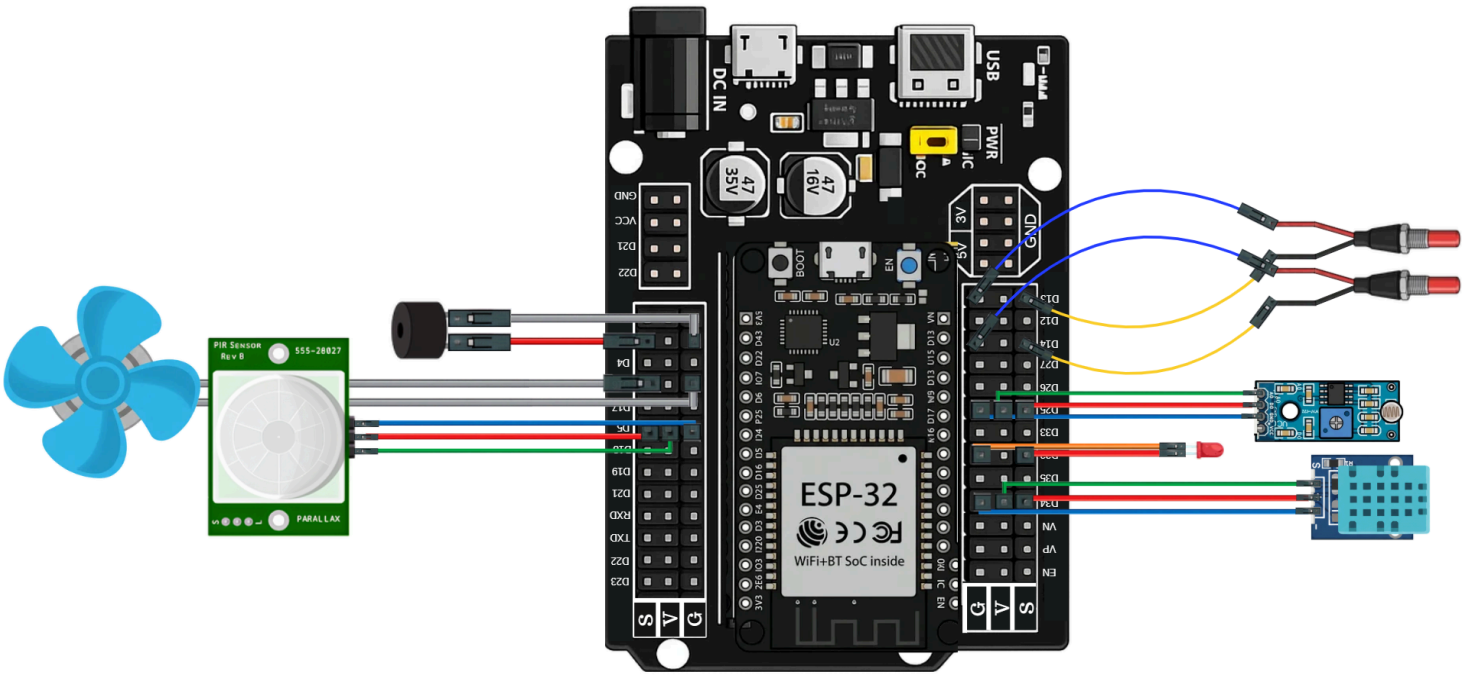
Component / Signal	Connection
PIR OUT	GPIO 5
LDR AO	GPIO 25
DHT11 DATA	GPIO 26
Fan Motor Control	GPIO 16 through shield/driver
Classroom Light LED	GPIO 33
Light Override Button	GPIO 14 to GND
Fan Override Button	GPIO 13 to GND
Buzzer	GPIO 2
LCD VCC	5V
LCD GND	GND
LCD SDA	GPIO 21
LCD SCL	GPIO 22
LCD Address	0x27

C. LIBRARIES REQUIRED

Wire • LiquidCrystal_I2C • DHT sensor library

NOTE: Connect the DC motor through the ESP32 shield motor output/driver or a transistor/MOSFET stage; do not drive a motor directly from a GPIO pin.

D. CIRCUIT DIAGRAM



PROJECT 20: Smart Classroom Light and Fan Automation System

CODE

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
DHT dht(34, DHT11);
const int PIR = 5, LDR = 25, FAN = 16, LIGHT = 32, B1 = 13, B2 = 14, BUZ = 2;
bool lightMan = false, fanMan = false, last1 = 1, last2 = 1;
void setup() {
  pinMode(PIR, INPUT);
  pinMode(FAN, OUTPUT);
  pinMode(LIGHT, OUTPUT);
  pinMode(B1, INPUT_PULLUP);
  pinMode(B2, INPUT_PULLUP);
  pinMode(BUZ, OUTPUT);
  dht.begin();
  lcd.init();
  lcd.backlight();
}
void loop() {
  bool b1 = digitalRead(B1), b2 = digitalRead(B2);
  if (last1 && !b1) {
    lightMan = !lightMan;
    delay(200);
  }
  if (last2 && !b2) {
    fanMan = !fanMan;
    delay(200);
  }
  last1 = b1;
  last2 = b2;
  bool occupied = digitalRead(PIR);
  int light = analogRead(LDR);
  float t = dht.readTemperature();
  bool lamp = lightMan || (occupied && light < 1800);
  bool fan = fanMan || (occupied && !isnan(t) && t > 29);
  digitalWrite(LIGHT, lamp);
  digitalWrite(FAN, fan);
  lcd.setCursor(0, 0);
  lcd.print("T:");
```

```
  lcd.print(t, 1);
  lcd.print(" L:");
  lcd.print(light);
  lcd.print(" ");
  lcd.setCursor(0, 1);
  lcd.print(lamp ? "LIGHT ON " : "LIGHT OFF");
  lcd.print(fan ? " FAN" : "    ");
  delay(250);
}
```